

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Develop An Integrated Application for Real-time
Location-based Flood Relief and Transparent
Volunteer Management**

VŨ HẢI ĐĂNG

Dang.VH226021@sis.hust.edu.vn

Program: Global ICT

Supervisor: PhD. Dinh Thi Ha Ly

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Develop An Integrated Application for Real-time
Location-based Flood Relief and Transparent
Volunteer Management**

VŨ HẢI ĐĂNG

Dang.VH226021@sis.hust.edu.vn

Program: Global ICT

Supervisor: PhD. Dinh Thi Ha Ly

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my teacher, Ph.D. Dinh Thi Ha Ly, at the School of Information and Communication Technology (SOICT), Hanoi University of Science and Technology, for her invaluable guidance and support throughout this project.

My heartfelt thanks go to my family for their endless love, and to my lover for always being my greatest source of encouragement during stressful times. I am also deeply grateful to my friends for their camaraderie and shared moments that made this academic journey truly memorable.

Finally, I want to thank myself. I am incredibly proud of my own hard work, resilience, and determination to overcome countless challenges to achieve the best results. Pushing through late nights of coding and research was demanding, but this milestone is a testament to my personal growth and unwavering dedication to my goals.

ABSTRACT

Floods are among the most frequent natural disasters in Vietnam, particularly in the central region, where they often disrupt communication and hinder rescue operations. Existing disaster assistance applications provide limited support for real-time location sharing, emergency coordination, and transparent management of post-disaster relief activities. These limitations reduce the effectiveness of rescue efforts and make it difficult to coordinate assistance among citizens, rescuers, benefactors, authorities, and administrators.

To address these challenges, this thesis presents FloodHelper, a disaster assistance platform that integrates real-time location sharing, emergency rescue coordination, and charity campaign management within a unified system. The proposed solution consists of a mobile application for end users and a web-based management system for authorities and administrators. The system adopts a hybrid software architecture combining the Client–Server model and the Publish–Subscribe communication pattern. The core technical approach utilizes the MQTT protocol for communication and location sharing, selected specifically for its lightweight footprint and high resilience in weak network environments. To mitigate intermittent connectivity, the mobile application implements a mechanism to save the victim’s last known location, ensuring rescue teams always have a starting reference point. In addition, the system provides a charity management module that supports campaign management, donation tracking, relief distribution, and financial reporting to improve transparency throughout the relief process.

The developed system successfully implements the major functional requirements identified during the analysis phase, including user management, friend management, real-time location sharing, emergency rescue coordination, charity campaign management, and administrative functions. The implementation demonstrates the feasibility of integrating these services into a single platform that supports multiple stakeholders involved in disaster response and post-disaster recovery.

The main contribution of this thesis is the development of an integrated disaster assistance platform that combines emergency communication with transparent charity management in a unified architecture. By facilitating timely information sharing, improving coordination among different user groups, and enhancing the transparency of relief activities, the proposed system provides a practical foundation for supporting disaster response and community recovery.

Student
Đặng
Vũ Hải Đăng

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and scope of the graduation thesis	1
1.3 Tentative solution	2
1.4 Thesis organization	3
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	4
2.1 Status survey	4
2.1.1 Analysis of existing systems and similar applications	4
2.1.2 Functional requirements of the proposed system	5
2.2 Functional overview	7
2.2.1 General use case diagram.....	7
2.2.2 Decompose use case "manage my account"	8
2.2.3 Decompose use case "interact with map"	8
2.2.4 Decompose use case "make friend"	9
2.2.5 Decompose use case "manage charity campaign"	10
2.2.6 Decompose use case "manage distress signal"	12
2.2.7 Decompose use case "manage user"	13
2.2.8 Decompose use case "review role requests"	14
2.2.9 Decompose use case "review updating-profile requests"	14
2.2.10 Business process	15
2.3 Functional description.....	18
2.3.1 Description of use case "Broadcast signal"	18
2.3.2 Description of use case "Handle signal"	19
2.3.3 Description of use case "Revoke signal"	20

2.3.4 Description of use case "Publish public announcement"	21
2.4 Non-functional requirement.....	22
CHAPTER 3. METHODOLOGY.....	23
3.1 Frontend framework: Flutter	23
3.2 Backend framework: NestJS	23
3.3 Database management: PostgreSQL	23
3.4 Object-relational mapping: Prisma ORM.....	24
3.5 Real-time communication: MQTT and EMQX cloud.....	24
3.6 Support services and infrastructure.....	25
3.6.1 Cloud storage: Cloudinary	25
3.6.2 Push notifications: Firebase Cloud Messaging	25
3.6.3 In-memory caching: Redis	25
3.6.4 Containerization and hosting: Docker and Render	25
3.7 Development tools	26
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	27
4.1 Architecture design.....	27
4.1.1 Software architecture selection.....	27
4.1.2 Overall design.....	28
4.1.3 Detailed package design	30
4.2 Detailed design.....	33
4.2.1 User interface design.....	33
4.2.2 Layer design	38
4.2.3 Database design	47
4.3 Application building	50
4.3.1 Libraries and tools	50
4.3.2 Achievement.....	54

4.3.3 Illustration of main functions	59
4.4 Testing.....	64
4.4.1 Testing methodology and techniques.....	64
4.4.2 Test case design for core functions.....	65
4.4.3 Summary of test results	66
4.4.4 Retrospective: failure analysis and resolutions	66
CHAPTER 5. SOLUTION AND CONTRIBUTION	68
5.1 Key contributions	68
5.1.1 MQTT topic architecture and communication mechanism for emer- gency rescue operations	68
5.1.2 Charity campaign lifecycle state management.....	71
5.2 Limitations of the work	72
CHAPTER 6. CONCLUSION AND FUTURE WORK	73
6.1 Conclusion.....	73
6.2 Future work.....	73

LIST OF FIGURES

Figure 2.1	General usecase diagram	7
Figure 2.2	Decompose use case "manage my account"	8
Figure 2.3	Decompose use case "interact with map"	9
Figure 2.4	Decompose use case "make friend"	10
Figure 2.5	Decompose use case "manage charity campaign"	11
Figure 2.6	Decompose use case "manage distress signal"	12
Figure 2.7	Decompose use case "manage user"	13
Figure 2.8	Decompose use case "review role requests"	14
Figure 2.9	Decompose use case "review updating-profile requests" . . .	15
Figure 2.10	Business process "Update personal information"	16
Figure 2.11	Bussiness process "Broadcast signal and rescue victim" . . .	17
Figure 2.12	Business process "Create and manage charity campaign" . .	18
Figure 4.1	Software Architecture	27
Figure 4.2	Layer Architecture for Frontend and Backend	28
Figure 4.3	General package diagram	29
Figure 4.4	Package diagram for backend	30
Figure 4.5	Package diagram for frontend	30
Figure 4.6	Detailed package design diagram for rescue module	31
Figure 4.7	Mockup for Home Screen	36
Figure 4.8	Mockup for Charity Campaign Management	37
Figure 4.9	Mockup for Auhtority Screen	38
Figure 4.10	Sequence diagram for use case "Broadcast signal" on the server side	46
Figure 4.11	Sequence diagram for use case "Handle signal" on the server side	47
Figure 4.12	Entity-relationship diagram	48
Figure 4.13	Database schema	49
Figure 4.14	Home screen	59
Figure 4.15	Broadcast signal for user	61
Figure 4.16	Signal list for rescuer	62
Figure 4.17	View campaigns for everyone	63
Figure 4.18	Campaign management for benefactor	64
Figure 5.1	Topic network for location sharing among friends	69

Figure 5.2	Topic network for location sharing between victims and res-	
	cuers	70
Figure 5.3	Finite state machine diagram for charity campaign	71

LIST OF TABLES

Bảng 2.1	Comparison of existing applications and the proposed system	5
Bảng 2.2	Use case specification for "Broadcast signal"	18
Bảng 2.3	Use case specification for "Handle signal"	19
Bảng 2.4	Use case specification for "Revoke signal"	20
Bảng 2.5	Use case specification for "Publish public announcement" . .	21
Bảng 4.1	Target device specifications	33
Bảng 4.2	SignalController - Attribute Design	38
Bảng 4.3	SignalController - Operation Design	39
Bảng 4.4	SignalService - Attribute Design	40
Bảng 4.5	SignalService - Operation Design	40
Bảng 4.6	SignalRepository - Attribute Design	43
Bảng 4.7	SignalRepository - Operation Design	43
Bảng 4.8	List of languages, libraries, APIs, and tools used in the entire system	50
Bảng 4.9	Details of packaged software products	55
Bảng 4.10	Summary of source code and packaged product statistics . . .	59
Bảng 4.11	Representative core test cases summary	65
Bảng 4.12	Summary of test executions and pass rates	66

LIST OF ABBREVIATIONS

Abriviation	Full Expression
ACID	Atomicity, Consistency, Isolation, Durability
AES	Advanced Encryption Standard
API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
D2TP	Delay-Tolerant Transport Protocol
D2TS	Delay-Tolerant Transmission Scheduler
DTN	Delay/Disruption-Tolerant Networking
DTO	Data Transfer Object
FCM	Firebase Cloud Messaging
FSM	Finite State Machine
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MQTT	Message Queuing Telemetry Transport
MVVM	Model-View-ViewModel
ORM	Object-Relational Mapping
OSM	OpenStreetMap
OTP	One-Time Password
QoS	Quality of Service
RAM	Random Access Memory
RDBMS	Relational Database Management System
REST	Representational State Transfer
SDK	Software Development Kit
SOS	Save Our Souls (Emergency distress signal)

Abriviation	Full Expression
SQL	Structured Query Language
TTL	Time-To-Live
UDP	User Datagram Protocol
UML	Unified Modeling Language
VietQR	Quick Response payment standard for Vietnam
VND	Vietnamese Dong

CHAPTER 1. INTRODUCTION

1.1 Motivation

In recent years, Vietnam has faced increasingly severe and unpredictable natural disasters, particularly in the central region. The year 2025 was recorded as one of the most extreme years for meteorological anomalies, breaking numerous historical records. According to the Ministry of Agriculture and Rural Development, natural disasters in 2025 caused estimated economic damages of up to 97,000 billion VND, resulting in 419 casualties and missing persons. Extreme weather events included 21 storms and tropical depressions in the East Sea, alongside unprecedented rainfall, such as the 1,739mm recorded within 24 hours at the Bach Ma station [1]. These extreme floods cause widespread devastation, frequently destroying local telecommunications infrastructure and leading to prolonged power outages. Consequently, trapped victims are completely isolated in environments with intermittent or entirely disconnected Internet access. The inability to transmit distress signals or pinpoint exact locations severely delays rescue operations, directly threatening the survival of stranded individuals. Establishing a resilient communication method to track and locate victims in these harsh conditions is a critical necessity.

Alongside the immediate need for rescue operations, the post-disaster relief and charity mobilization process presents another significant societal challenge regarding financial transparency. Recently, the mobilization of relief funds by high-profile individuals and voluntary organizations has sparked widespread public skepticism. Instances where tens or hundreds of billions of VND were collected without independent auditing, detailed bank statements, or clear transaction records have led to profound crises of public trust. Furthermore, the lack of a standardized system allows for the unverified distribution of goods, while mandatory but untracked micro-donations in educational institutions highlight the systemic absence of an accountable framework. The current manual and fragmented approaches to managing charity campaigns fail to provide traceability. Therefore, developing a centralized platform to strictly manage the lifecycle of relief funds—from donation to final distribution—is essential to restore community trust and ensure resources reach the intended beneficiaries transparently and legally.

1.2 Objectives and scope of the graduation thesis

To overcome these limitations, this thesis aims to design and develop Flood-Helper, a comprehensive ecosystem combining offline-resilient rescue coordination and transparent charity management. The system's scope encompasses three

main interfaces tailored to specific user roles. First, a mobile application is developed for Victims, Rescuers, and Benefactors, facilitating location broadcasting and campaign interactions. Second, a web-based portal is designed specifically for Authorities to verify user roles, approve rescue requests, and monitor operations. The administrative dashboard for system-wide user management is considered as future work.

In terms of business logic, the charity management subsystem strictly enforces a transparent workflow. A Benefactor creates a campaign, which remains inactive until officially approved by an Authority. Once the donation phase begins, all financial contributions are publicly recorded. Subsequently, the allocation phase requires the Benefactor to publicly declare the distribution of funds and specific relief items. The campaign concludes only when legitimate bank statements are uploaded for final verification. To ensure absolute integrity, Authorities are granted the power to suspend any campaign exhibiting suspicious activities during any phase.

1.3 Tentative solution

To address the challenges of intermittent connectivity during floods, the proposed solution utilizes the Message Queuing Telemetry Transport (MQTT) protocol as the core communication foundation. MQTT is selected due to its lightweight payload, low bandwidth requirements, and robust message retention capabilities, making it exceptionally reliable in weak network conditions. On the client side, the mobile application leverages background isolates to maintain persistent background processes, periodically capturing and pushing the user's GPS coordinates to the MQTT broker without requiring active screen time. All critical actions, including broadcasting SOS signals, handling rescue assignments, and terminating alerts, are executed via MQTT topics, which subsequently synchronize with the backend server whenever the network connection is briefly restored.

For the charity management module, the backend architecture implements automated scheduling mechanisms, utilizing cron jobs to securely manage campaign state transitions (e.g., from "Approved" to "Donating" and "Distributing") based on predefined temporal constraints and verification rules, minimizing manual interference.

The primary contributions of this graduation thesis are threefold: (i) designing a resilient mobile architecture utilizing MQTT and background isolates to ensure continuous location tracking and SOS broadcasting in unstable network environments; (ii) digitizing the traditional charity process into a strictly state-driven, pub-

licly transparent workflow that enforces accountability from the initial donation to the final bank statement validation; and (iii) successfully integrating these two critical components into a unified, functional platform (FloodHelper) to comprehensively support both immediate disaster rescue and post-disaster relief efforts.

1.4 Thesis organization

The remainder of this graduation thesis report is organized as follows.

Chapter 2 presents the requirement survey and analysis phase of the FloodHelper system. This chapter focuses on examining the practical needs of victims in flooded areas and evaluating the limitations of current rescue applications, thereby establishing detailed functional and non-functional requirements for both the rescue and charity management subsystems.

In Chapter 3, I introduce the methodology applied to resolve the identified challenges. The chapter details the selection of the MQTT protocol for communication in weak network environments and outlines the technological approaches utilized to build a robust architecture for data synchronization and transparent charity tracking.

Chapter 4 details the system design, implementation process, and practical evaluation. This chapter encompasses the overall architectural diagrams, database modeling, and application development, followed by testing results that assess the system's communication performance under simulated intermittent network conditions.

Chapter 5 summarizes the finalized solutions and the primary contributions of the thesis. The content highlights the successful integration of the resilient location-sharing mechanism and the comprehensive charity management dashboard, demonstrating how these components directly address the initial research problems.

Finally, Chapter 6 provides the conclusion of the study and outlines future work. This chapter reflects on the achieved results, acknowledges the current limitations of the system, and proposes potential research directions, including the integration of Delay-Tolerant Networking (DTN) to support data sharing in completely disconnected scenarios.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

Chapter 2 presents a comprehensive survey and analysis of the system requirements. The chapter begins with a status survey to establish the project context, followed by a functional overview that illustrates the system's scope through general and detailed use case diagrams as well as business processes. Subsequently, specific features are examined in depth in the functional description section. Finally, the chapter concludes by outlining the essential non-functional requirements necessary to ensure the system's performance and quality standards.

2.1 Status survey

A thorough survey of the current landscape is essential to identify the precise software requirements and ensure the proposed solution effectively addresses practical challenges. This survey analyzes data from three primary sources: target users in flood-prone areas, existing emergency response systems, and similar location-sharing applications on the market.

2.1.1 Analysis of existing systems and similar applications

To identify the functional requirements of the proposed system, several existing applications related to communication, location sharing, emergency rescue, and charity management were studied. The selected systems include Zalo [2], a web-based SOS platform [3], and Whoo [4], as they represent different approaches to location sharing and emergency support.

Zalo - instant messaging application: Zalo [2] is the most popular communication tool in Vietnam. Users can share their live location with contacts. However, this feature is strictly session-based and requires manual activation within a specific chat box. Crucially, it lacks continuous background tracking and entirely fails when the Internet connection drops. It also does not provide a centralized mechanism to broadcast SOS signals directly to authorized rescue teams.

"An toàn cứu hộ" - web-based SOS platform: It is dedicated platforms designed to transmit distress signals. While directly connected to rescue operations, its primary limitation is their reliance on web browsers, meaning they operate strictly online. If a victim loses network connectivity, the system immediately drops the connection, preventing any signal transmission. Furthermore, these platforms often lack features for victims to share their status with family members.

Whoo - social location app: Whoo [4] is a mobile application designed for sharing locations among friends. It successfully implements a mechanism to save the

user's last known location (last-location caching). However, it is primarily built for social networking rather than emergency response, meaning it cannot link victims with professional rescue units. Additionally, the application has been reported to suffer from synchronization instability.

Table 2.1 summarizes the comparison between these existing systems and the proposed FloodHelper application. The comparison shows that existing systems typically focus on only one aspect of disaster support. Messaging applications mainly provide communication functions, while emergency platforms emphasize incident reporting. None of the surveyed systems integrates emergency rescue coordination with charity campaign management into a unified platform. Therefore, this graduation project proposes an integrated system that combines these functions to better support flood disaster response and post-disaster relief activities.

Table 2.1: Comparison of existing applications and the proposed system

Feature	Zalo	Web SOS	Whoo	Proposed System
Platform	Mobile / PC	Web	Mobile	Mobile & Web
Offline Last-Location Saving	No	No	Yes	Yes
Continuous Background Tracking	No	No	Yes	Yes
Direct Rescue Team Integration	No	Yes	No	Yes
Transparent Charity Management	No	No	No	Yes

2.1.2 Functional requirements of the proposed system

Based on the survey of existing applications and the practical requirements of flood disaster response, the proposed FloodHelper system is designed to provide an integrated platform that supports emergency communication, rescue coordination, and charity campaign management. The major software features to be developed are described as follows.

a, User and role management

The system provides user registration, authentication, and profile management for different types of users, including Normal Users, Benefactors, Rescuers, Authorities, and Administrators. Depending on the assigned role, users are granted different permissions to access the corresponding system functions. The system also supports role application and profile verification workflows to ensure that privileged roles are approved by the responsible authorities before becoming active.

b, Friend connection and location sharing

The application enables users to establish friend connections and periodically share their current locations with connected friends while the application is running. Besides geographical coordinates, the shared information also reflects the user's current availability status, allowing friends to know whether the user is currently online or offline. To provide greater control over personal privacy, users can temporarily suspend location updates for selected friends, who will then only be able to view the user's last shared location until sharing is resumed.

c, Emergency rescue support

The system provides emergency communication functions that allow users to broadcast distress signals together with their current locations during flood situations. Rescue personnel can monitor active emergency requests, identify victims on a digital map, and update the handling status of each rescue request. Throughout the rescue process, location information is periodically synchronized so that both victims and rescuers can track rescue progress more effectively.

d, Charity campaign management

The system supports the complete lifecycle of charity campaigns, from campaign creation and approval to donation collection, relief distribution, and financial reporting. Benefactors can organize charity campaigns, while authorities review submitted campaigns before publication. During campaign execution, donors are able to monitor campaign progress, distribution activities, and financial statements to improve transparency.

e, Public information and notification

Authorities can publish public announcements regarding weather conditions, emergency instructions, evacuation guidance, or other disaster-related information. The application delivers these announcements to users located in the affected areas, allowing important information to be disseminated promptly. In addition, the system generates notifications for rescue activities, charity campaigns, and account-related events to keep users informed of important updates.

f, Administrative management

Administrative functions are provided to support overall system operation and supervision. Administrators can manage user accounts, while authorities review role applications, profile update requests, and charity campaign requests. These management functions help ensure that the platform operates in accordance with the defined policies and maintains the integrity of system data.

2.2 Functional overview

2.2.1 General use case diagram

Figure 2.1 presents the overall use case diagram of the FloodHelper system. In details, the system involves five primary actors, each responsible for different operational tasks.

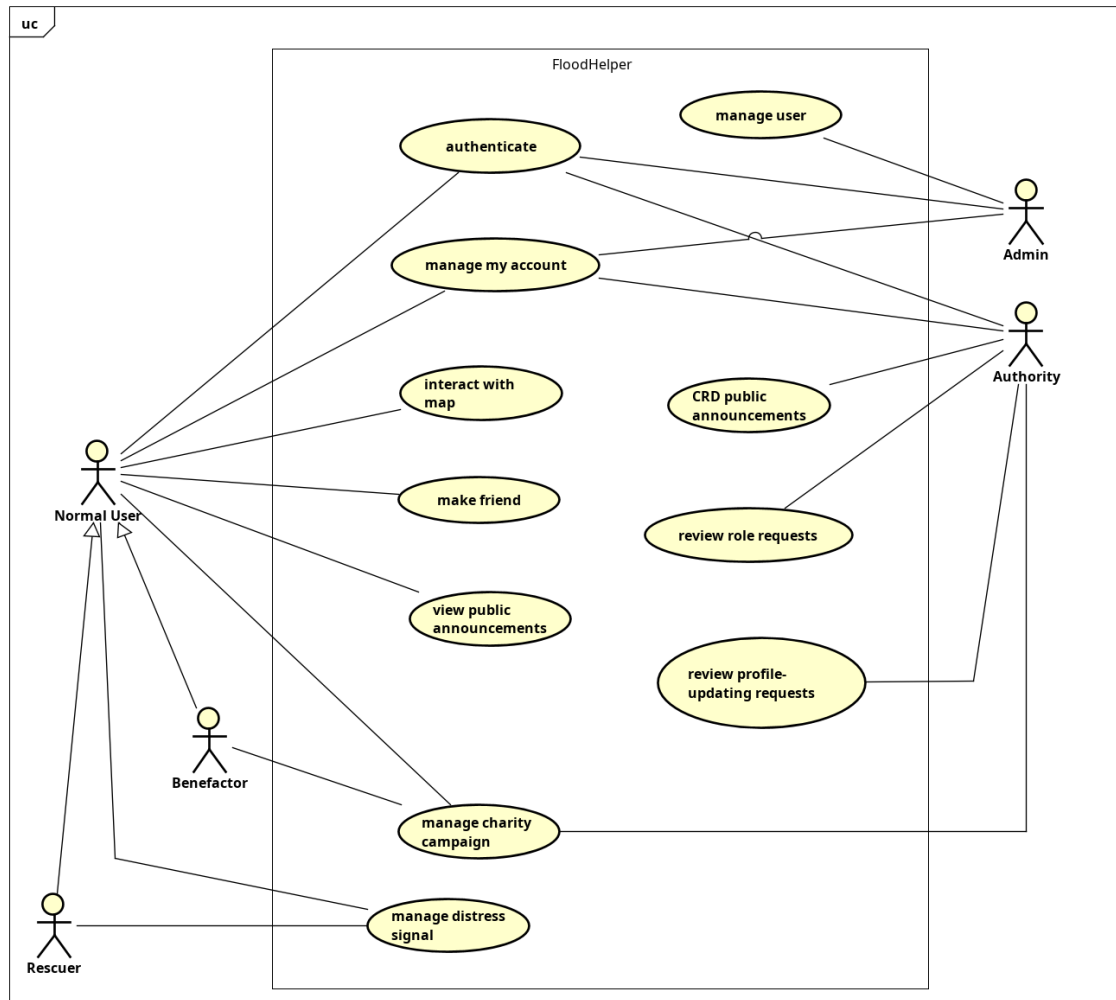


Figure 2.1: General usecase diagram

Normal user is the primary user of the mobile application. A normal user can manage personal information, connect with friends, share locations, broadcast emergency distress signals, participate in charity campaigns, and receive public announcements.

Benefactor represents individuals or organizations responsible for organizing charity campaigns. In addition to the functions available to normal users, benefactors can create, manage, and update charity campaigns.

Rescuer is responsible for monitoring emergency requests and coordinating rescue activities. Rescuers receive distress signals, locate victims, and update rescue

statuses during emergency operations.

Authority supervises rescue and charity activities. Authorities review campaign requests, approve user role applications, verify profile updates, and publish public announcements.

Administrator manages system operation by maintaining user accounts and administrative data to ensure stable platform operation.

2.2.2 Decompose use case "manage my account"

Figure 2.2 presents the account management module of the FloodHelper system. The module provides common account functions, including registration, profile management, password management, and role application. While all users can manage their personal accounts, users with specialized roles are required to submit profile update requests for verification before changes are applied. This mechanism helps maintain the reliability of verified user information throughout the system.

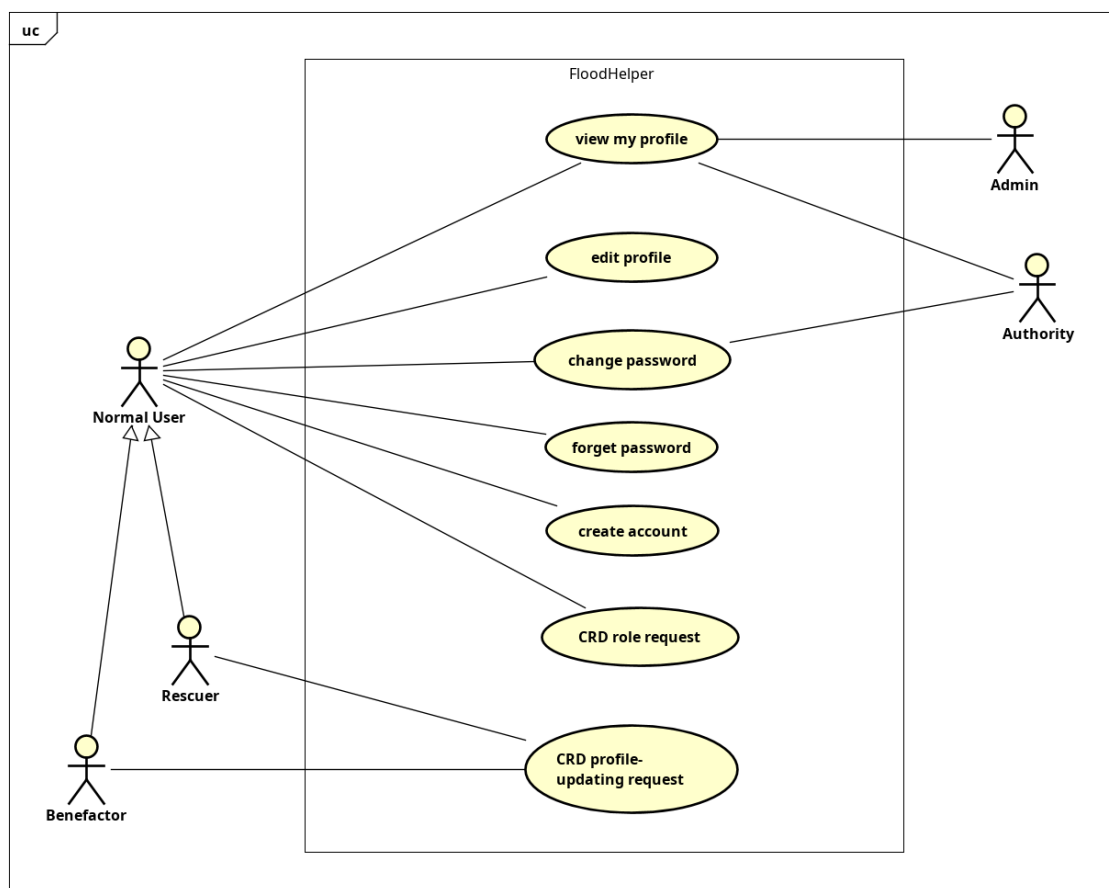


Figure 2.2: Decompose use case "manage my account"

2.2.3 Decompose use case "interact with map"

Figure 2.3 illustrates the use cases related to map interaction, which are available to the Normal User. This module provides functionalities for map navigation,

location visualization, and privacy management.

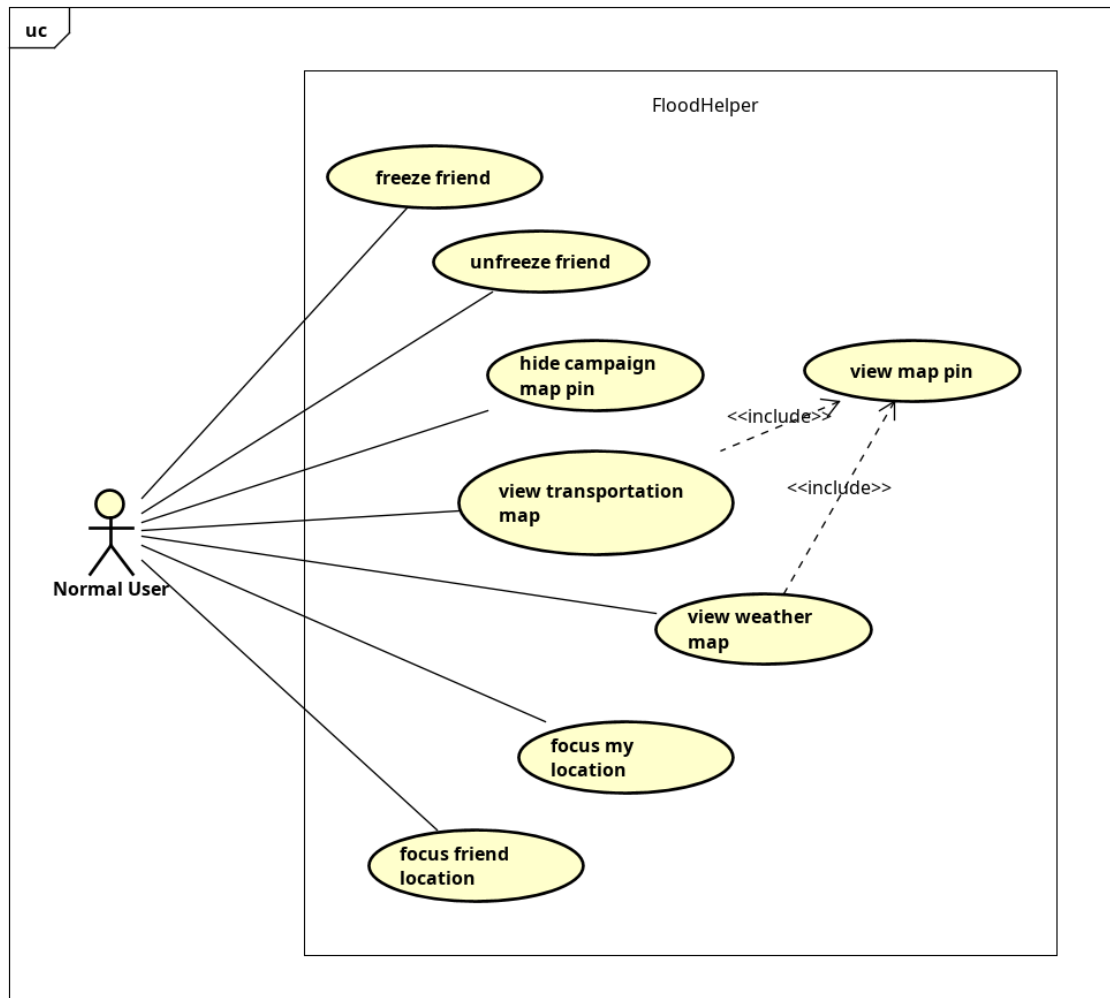


Figure 2.3: Decompose use case "interact with map"

Users can access both the "View transportation map" and the "View weather map" to obtain geographical and environmental information. In both cases, the corresponding map pins are displayed to present relevant locations and events on the map.

To support navigation, users can focus the map on their current location or on the location of a selected friend. The system also allows users to manage the visibility of shared information by freezing or unfreezing location sharing for specific friends. In addition, users may hide charity campaign map pins to reduce visual clutter and improve the readability of the map interface.

2.2.4 Decompose use case "make friend"

Figure 2.4 presents the use cases related to friend management for the Normal User. This module enables users to establish and maintain friend relationships within the FloodHelper system.

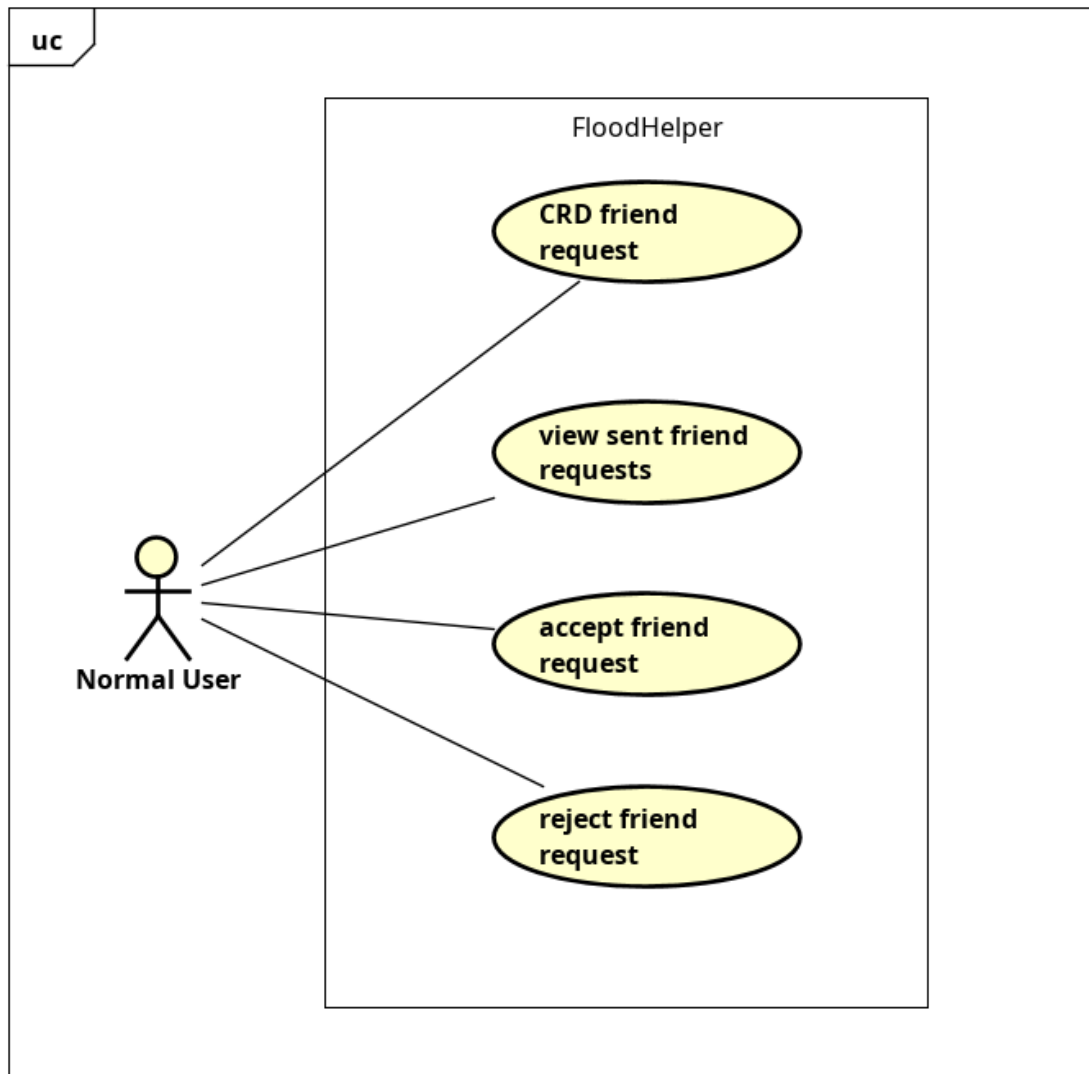


Figure 2.4: Decompose use case "make friend"

Users can search for other users and send friend requests to initiate a connection. The system also allows users to view pending and previously sent friend requests, as well as cancel requests that have not yet been processed. When receiving a friend request, users may either accept or reject the request. In addition, existing friendships can be removed when users no longer wish to share information with each other.

These functionalities establish the social network required for subsequent features such as location sharing, emergency communication, and rescue coordination.

2.2.5 Decompose use case "manage charity campaign"

Figure 2.5 illustrates the use cases related to charity campaign management, involving three primary actors: Benefactor, Normal User, and Authority.



Figure 2.5: Decompose use case "manage charity campaign"

The Benefactor is responsible for organizing and managing charity campaigns. Benefactors can create, update, and delete campaign information, submit campaign requests for approval, publish campaign announcements, manage distribution items, and upload bank statements after the campaign is completed. These functions support the management of a charity campaign throughout its lifecycle.

The Normal User participates as a donor and can access campaign information through various viewing functions, including campaign details, announcements, distribution records, transaction history, and bank statements. Users can also donate to active campaigns and view campaign locations on the map to better understand the distribution of relief activities.

The Authority supervises charity campaigns by reviewing submitted campaign requests before publication. Authorities can approve or reject campaign requests and suspend active campaigns when necessary to ensure that charity activities comply with the platform's management policies.

2.2.6 Decompose use case "manage distress signal"

Figure 2.6 presents the use cases related to emergency distress signal management, involving two primary actors: the Normal User and the Rescuer.

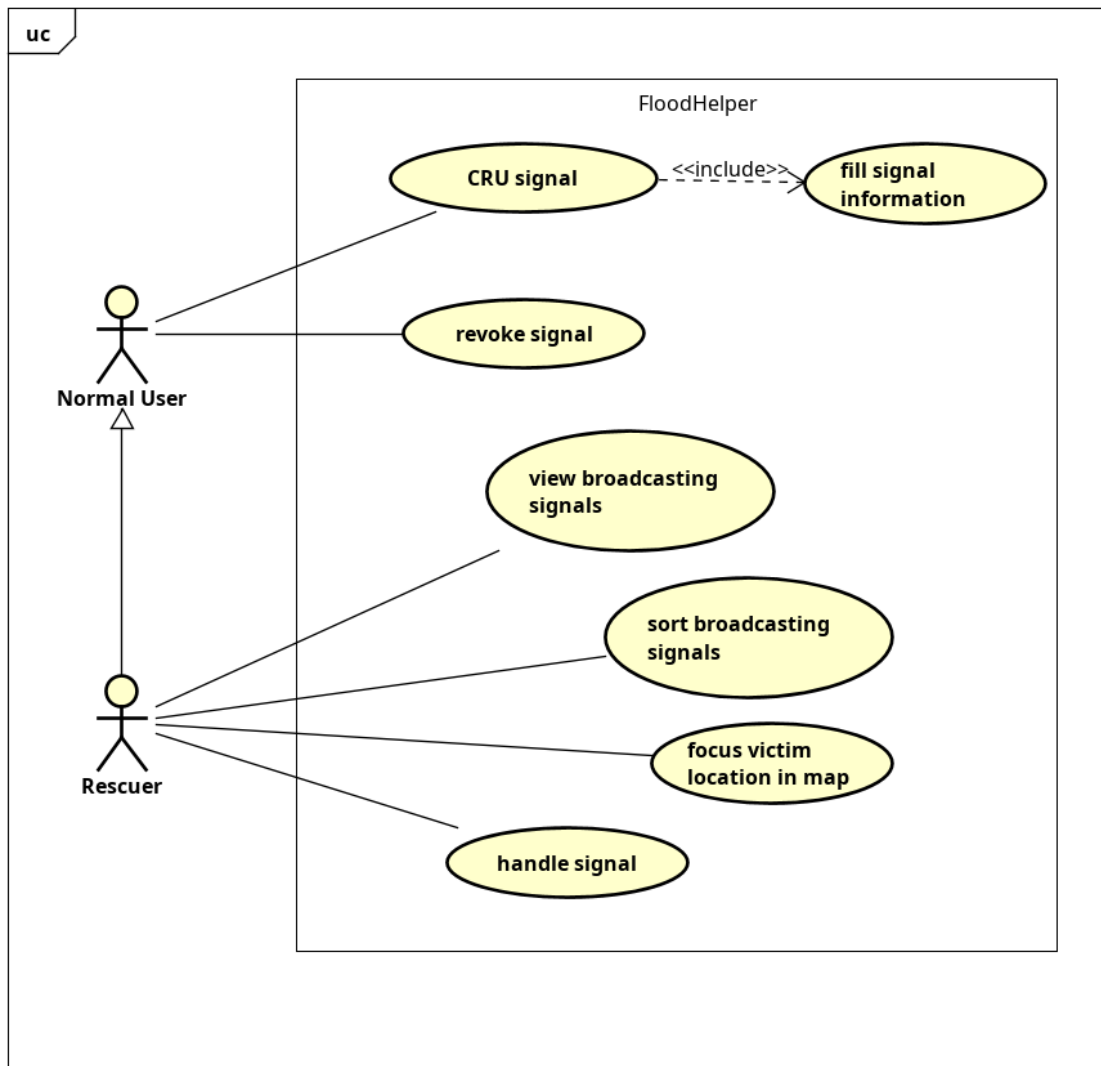


Figure 2.6: Decompose use case "manage distress signal"

The Normal User can create, view, and update distress signals when emergency assistance is required. During signal creation, users provide essential information describing the emergency situation and their current location to facilitate rescue operations. If assistance is no longer needed, users may revoke an active distress signal before it is handled by a rescuer.

The Rescuer is responsible for monitoring and responding to active distress signals. Rescuers can view and sort broadcasting signals to identify and prioritize emergency requests, locate victims on the map, and handle selected distress signals throughout the rescue process. These functions support effective rescue coordination by enabling rescuers to monitor ongoing incidents and respond to emergency

requests in a timely manner.

2.2.7 Decompose use case "manage user"

Figure 2.7 presents the use cases related to user management, which are performed exclusively by the Admin. This module provides administrative functions for managing user accounts and maintaining system operation.

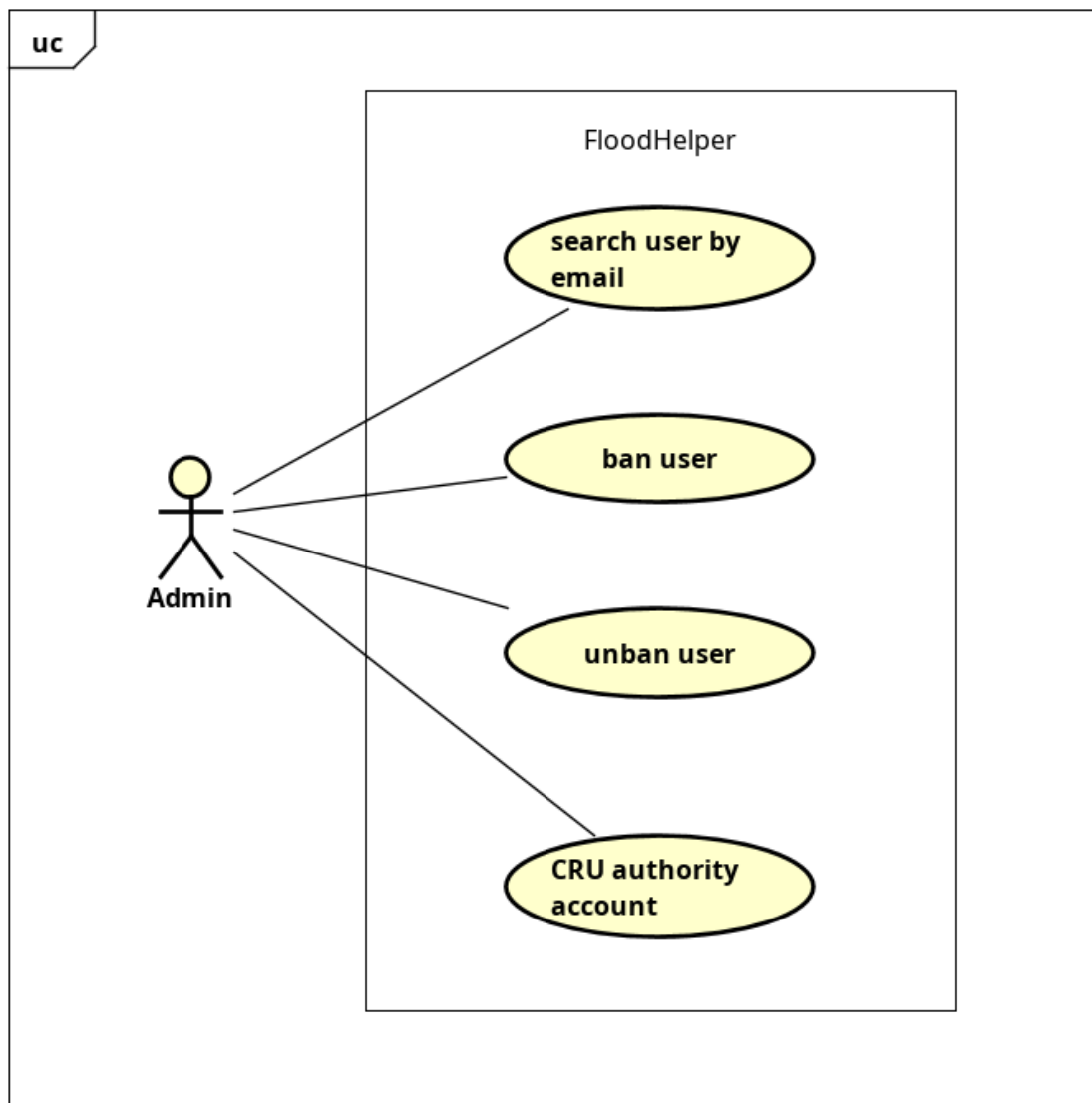


Figure 2.7: Decompose use case "manage user"

The Admin can search for users by email to quickly locate specific accounts within the system. To enforce platform policies and manage inappropriate user activities, the Admin is authorized to ban or unban user accounts when necessary. In addition, the Admin is responsible for managing Authority accounts by creating, viewing, and updating their information. These functions support the administration of user access and ensure that authorized personnel are properly maintained within the system.

2.2.8 Decompose use case "review role requests"

Figure 2.8 presents the use cases related to role request management, which are performed by the Authority. This module enables the Authority to review requests submitted by users who wish to obtain specialized roles within the FloodHelper system.

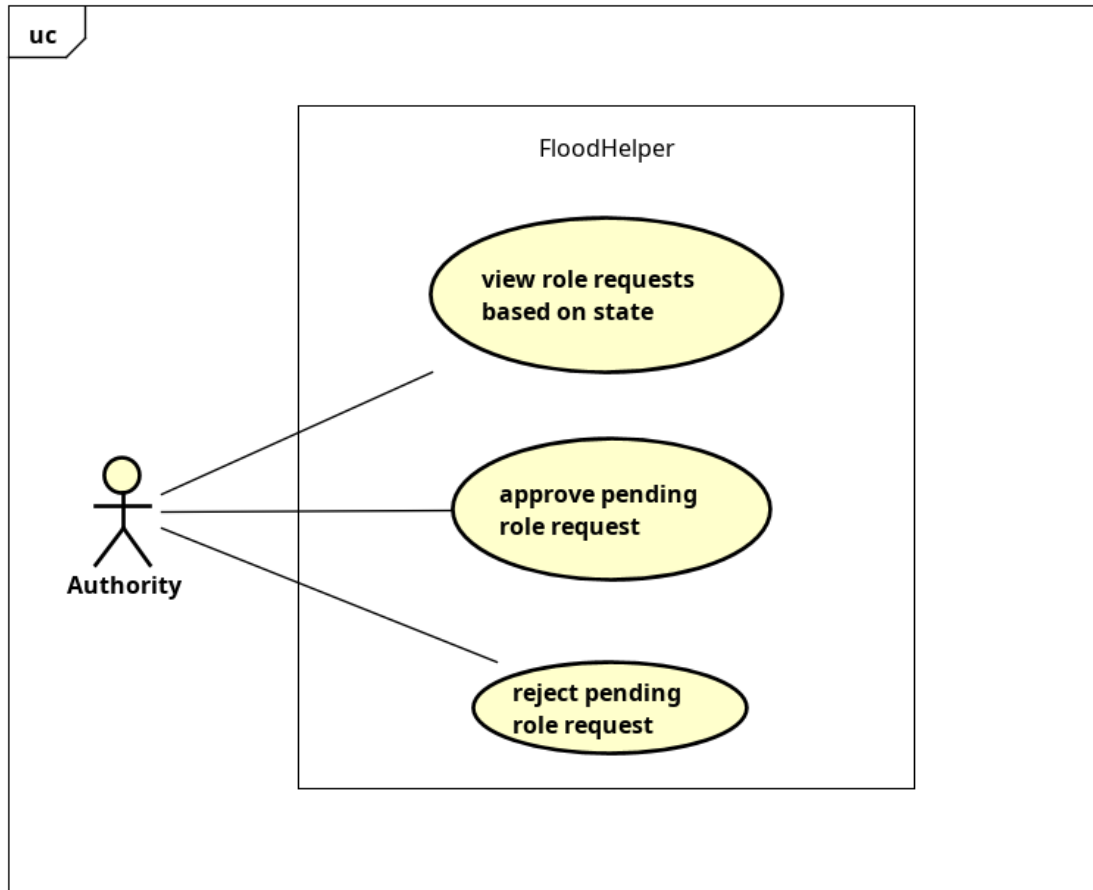


Figure 2.8: Decompose use case "review role requests"

The Authority can view role requests based on their current processing status to facilitate the review process. After evaluating each request, the Authority may either approve or reject the pending role request. An approved request grants the applicant the corresponding system role, while a rejected request prevents the requested role from being assigned.

These functions ensure that specialized roles, such as Benefactor and Rescuer, are assigned only after verification by the responsible authority.

2.2.9 Decompose use case "review updating-profile requests"

Figure 2.9 presents the use cases related to profile-updating request management, which are performed by the Authority. This module enables the Authority

to review profile modification requests submitted by users with specialized roles, such as Benefactors and Rescuers.

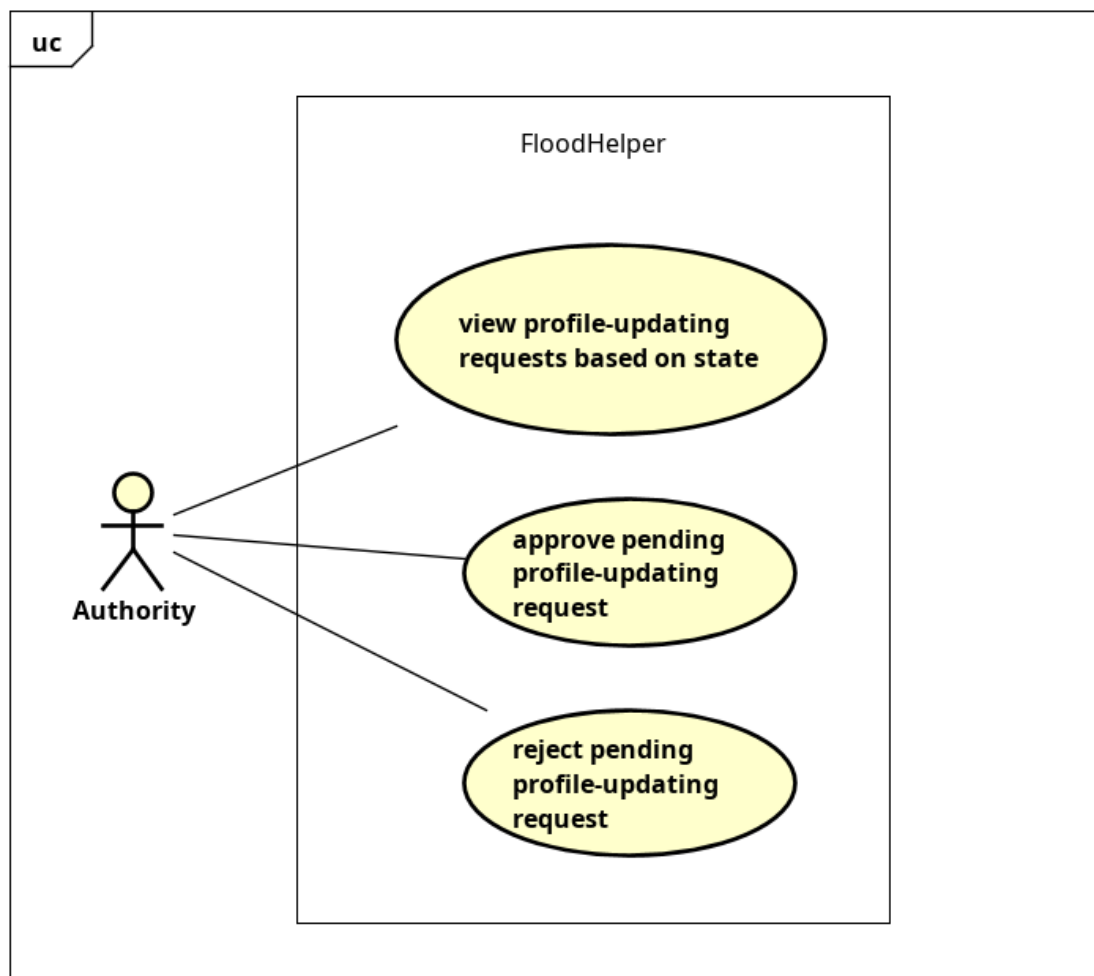


Figure 2.9: Decompose use case "review updating-profile requests"

The Authority can view profile-updating requests based on their current processing status to facilitate the review process. After examining the submitted information, the Authority may either approve or reject each pending request. Approved requests update the corresponding user profile with the verified information, while rejected requests leave the existing profile unchanged.

These functions help ensure that the information displayed for specialized users remains accurate and verified before being made available within the system.

2.2.10 Business process

The FloodHelper system includes several core business processes that involve interactions among multiple actors to accomplish major system functionalities. These processes describe the sequence of activities performed by users and the system in typical operational scenarios, providing a high-level view of how differ-

ent functional modules collaborate to achieve the intended objectives.

To illustrate these workflows, business process diagrams are used to represent the interactions between actors and the system. In this project, three representative business processes are presented: Update Personal Information, Broadcast Signal and Rescue Victim, and Create and Manage Charity Campaign. These processes cover the key operational scenarios of user profile management, emergency rescue coordination, and charity campaign management within the FloodHelper system.

The following subsections describe each business process in detail.

a, Update personal information

Figure 2.10 illustrates the business process for updating personal information. The process begins when a user submits a request to modify their profile information. Depending on the user's role, the submitted information may be updated directly or require verification by the Authority before the changes are applied. This process ensures that profile information remains accurate while maintaining the reliability of verified user accounts.

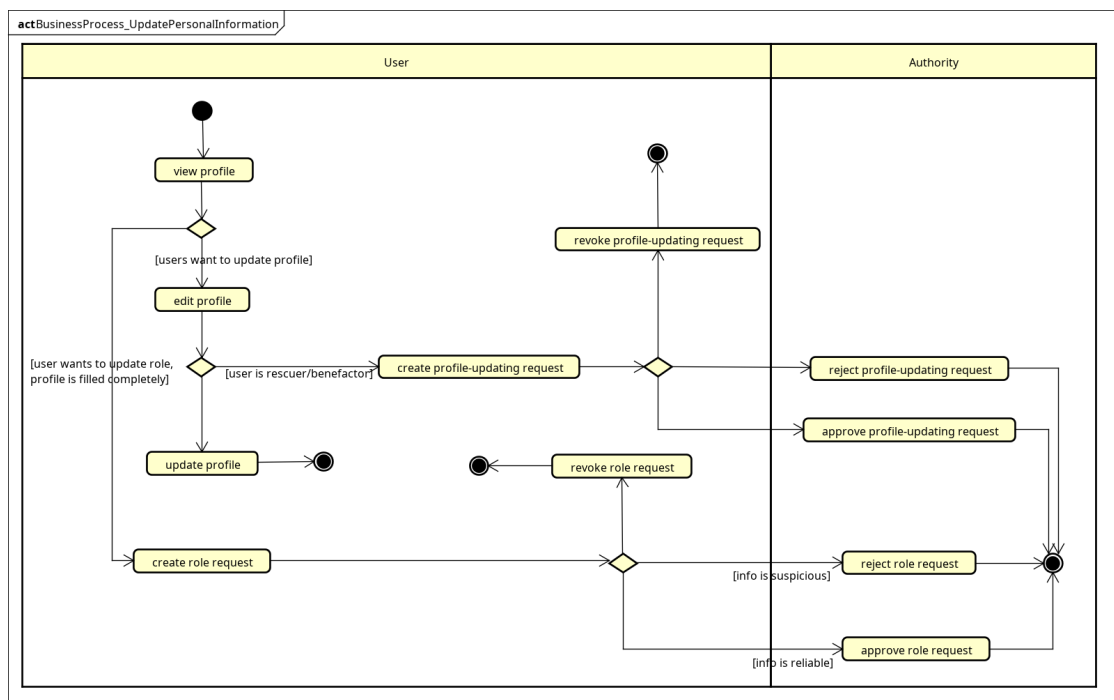


Figure 2.10: Business process "Update personal information"

b, Broadcast signal and rescue victim

Figure 2.11 presents the business process for emergency rescue coordination. The workflow starts when a user broadcasts a distress signal containing the required emergency information and current location. The system makes the distress signal available to rescuers, who can monitor active requests, locate the victim, and handle the assigned rescue task. The process concludes when the rescue request is

completed or the distress signal is revoked.

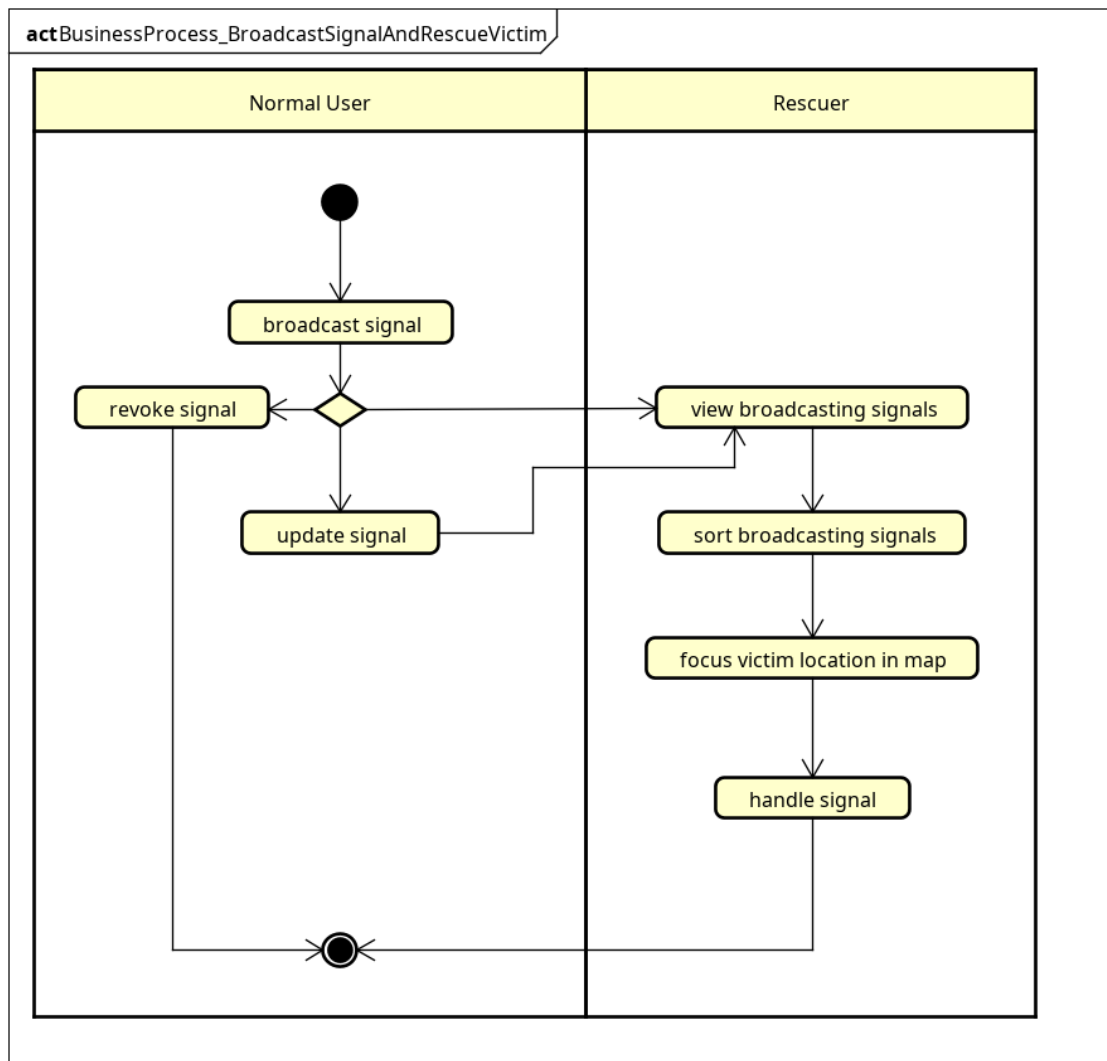


Figure 2.11: Bussiness process "Broadcast signal and rescue victim"

c, Create and manage charity campaign

Figure 2.12 illustrates the business process for charity campaign management. The workflow begins with a Benefactor creating a charity campaign and submitting it for approval. After the campaign is approved by the Authority, it becomes available for public donations. Throughout the campaign, the Benefactor manages announcements, distribution activities, and financial reports, while donors can contribute to the campaign and monitor its progress until the campaign is completed.

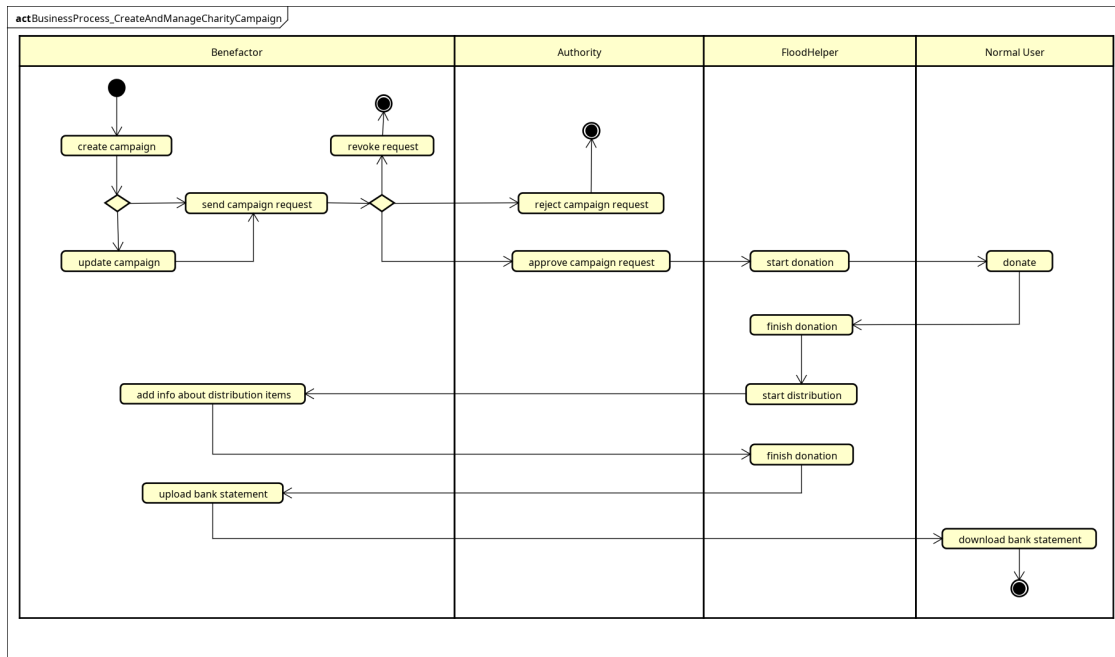


Figure 2.12: Business process "Create and manage charity campaign"

2.3 Functional description

2.3.1 Description of use case "Broadcast signal"

Table 2.2 presents the detailed specification of the use case "broadcast signal".

Table 2.2: Use case specification for "Broadcast signal"

Use case name	Broadcast signal
Use case ID	UC001-BS
Actors	Normal user, Rescuer
Pre-conditions	- The user has successfully logged into the system. - The user's location service is available. - The user has not created another active distress signal.
Main flow	1. The user selects the Broadcast Distress Signal function. 2. The system displays the distress signal form. 3. The user enters the required emergency information. 4. The user submits the distress signal. 5. The system validates the submitted information. 6. The system creates a new distress signal and records its information.

Continued on next page

Table 2.2 – Continued from previous page

Alternative and exception flows	-Alternative flow 3.1: At Step 3, the user does not enter any emergency information. The system applies default values for the distress signal and continues with Step 4. -Alternative flow 4.1: At any time before Step 4, the user cancels the operation. The system discards the entered information and returns to the previous screen.
Post-conditions	- A new distress signal is successfully created. - The distress signal is available for rescuers to monitor. - The user can track the status of the submitted distress signal.

2.3.2 Description of use case "Handle signal"

Table 2.3 presents the detailed specification of the use case "handle signal".

Table 2.3: Use case specification for "Handle signal"

Use case name	Handle signal
Use case ID	UC002-HS
Actors	Rescuer
Pre-conditions	- The Rescuer has successfully logged into the system. - The Rescuer is viewing the map with active distress signals. - A victim has broadcasted an active distress signal.
Main flow	1. The Rescuer selects an active distress signal on the map. 2. The system displays the details of the selected distress signal. 3. The Rescuer confirms handling the distress signal. 4. The system validates the request and updates the status of the distress signal to "Being handled". 5. The system notifies the victim that their distress signal is being handled. 6. The system updates the map view for other rescuers to show that the signal is being handled.

Continued on next page

Table 2.3 – Continued from previous page

Alternative and exception flows	-Alternative flow 3.1: At Step 3, the Rescuer decides not to handle the signal and cancels the operation. The system returns to the map view. -Alternative flow 5.1: At Step 5, if the victim is also registered as a rescuer, the system updates their personal signal status while maintaining their access to the rescue coordination map.
Post-conditions	- The distress signal status is successfully updated to "Being handled". - The victim is notified of the rescue progress. - Other rescuers are updated to avoid duplicate rescue deployments.

2.3.3 Description of use case "Revoke signal"

Table 2.4 presents the detailed specification of the use case "revoke signal".

Table 2.4: Use case specification for "Revoke signal"

Use case name	Revoke signal
Use case ID	UC003-RS
Actors	Normal user, Rescuer
Pre-conditions	- The user has successfully logged into the system. - The user has an active distress signal.
Main flow	1. The user selects the option to revoke their active distress signal. 2. The system requests confirmation from the user. 3. The user confirms the revocation. 4. The system validates the request and marks the distress signal as inactive. 5. The system stops broadcasting the distress signal. 6. The system notifies rescuers in the area that the distress signal is no longer active.

Continued on next page

Table 2.4 – Continued from previous page

Alternative and exception flows	-Alternative flow 3.1: At Step 3, the user decides to cancel the revocation. The system returns the user to the active signal details view.
Post-conditions	- The distress signal status is successfully marked as inactive. - Rescuers in the area are updated, and the distress signal is cleared from their coordination maps.

2.3.4 Description of use case "Publish public announcement"

Table 2.5 presents the detailed specification of the use case "publish public announcement".

Table 2.5: Use case specification for "Publish public announcement"

Use case name	Publish public announcement
Use case ID	UC004-PAP
Actors	Authority
Pre-conditions	- The Authority is logged into the system. - The Authority has the permissions to publish announcements.
Main flow	1. The Authority selects the option to create a public announcement. 2. The system displays the announcement creation form. 3. The Authority enters the announcement details (such as title, content, and optional file attachments) and submits the form. 4. The system validates the input. 5. The system saves the public announcement. 6. The system notifies all users in the Authority's ward of the new announcement. 7. The system displays a confirmation message to the Authority and refreshes the announcements list.

Continued on next page

Table 2.5 – Continued from previous page

Alternative and exception flows	-Alternative flow 3.1: At Step 3, the Authority decides to cancel the creation. The system discards the input and returns to the previous screen. -Alternative flow 3.2: At Step 3, if the Authority attaches files, the system uploads and stores the files associated with the announcement.
Post-conditions	- The public announcement is successfully published and stored. - Users in the designated ward are notified. - The announcement list is updated for the Authority and users.

2.4 Non-functional requirement

Performance (Latency): Real-time location tracking and signal broadcasting via the MQTT broker must maintain a delivery latency of under 500ms to ensure accurate, up-to-date mapping of victims and rescuers.

Reliability (Network Resilience): The mobile application must utilize delay-tolerant networking strategies and background isolates to queue emergency signals and location data locally, ensuring eventual transmission even when operating in unstable or heavily degraded network environments.

Scalability (Concurrency): The containerized backend infrastructure must be capable of automatically scaling to handle sudden, massive spikes in concurrent connections and API requests during widespread emergency events without system degradation.

Maintainability (Architecture): The system's codebase must strictly adhere to clean architecture principles (such as the Model-View-ViewModel (MVVM) pattern for the frontend) to ensure straightforward unit testing, predictable state management, and the seamless integration of future communication protocols.

CHAPTER 3. METHODOLOGY

This section describes the technology stack, services, and development tools selected for the FloodHelper system. The selection is based on their suitability to meet specific system demands: real-time location sharing under unstable network conditions, transactional integrity for donation management, and cross-platform accessibility.

3.1 Frontend framework: Flutter

Flutter [5] is selected to develop the mobile application for both normal users (victims) and rescuers. In the FloodHelper system, the mobile app must handle real-time location updates, render interactive maps (utilizing OpenStreetMap [6]), and update UI states dynamically. Flutter is chosen because it renders UI components directly using its own graphic engine (Skia/Impeller). This direct rendering ensures smooth map interactions and continuous background rendering, which are critical when victims and rescuers need to track coordinates in emergency situations. Furthermore, Flutter enables development for both Android and iOS from a single Dart codebase, significantly reducing development and maintenance overhead.

3.2 Backend framework: NestJS

The backend server of the system is built using NestJS [7], a progressive Node.js [8] framework. The backend is responsible for processing REST API requests, coordinating real-time message routing, and managing transactional workflows such as role applications and charity campaigns. NestJS is selected over Express.js [9] because of its opinionated, modular architecture (modules, controllers, and services). This structure prevents code degradation as the system grows and ensures that critical donation logic is separated from authentication and notification systems. Additionally, NestJS's native support for TypeScript provides compile-time type safety, minimizing runtime exceptions in processing financial records and emergency states.

3.3 Database management: PostgreSQL

PostgreSQL [10] is chosen as the primary relational database to store persistent records, including user profiles, emergency signal histories, and campaign transaction logs. In a system managing charitable donations, transactional reliability is paramount. PostgreSQL's strict ACID (Atomicity, Consistency, Isolation, Durability) compliance guarantees that donation operations are recorded accurately with-

out data corruption during concurrent write-heavy events. Compared to MySQL [11], PostgreSQL offers superior support for complex queries and extensibility for potential geospatial indexing, which is necessary for geographic search operations in flood areas.

3.4 Object-relational mapping: Prisma ORM

To interact with the database, the backend utilizes the Prisma [12] Object-Relational Mapper (ORM) instead of TypeORM [13]. Prisma is selected because it uses a centralized, human-readable schema file ('schema.prisma') as the single source of truth, automating database migrations. Prisma auto-generates a strongly-typed database client, ensuring that database queries in NestJS are validated at compile-time, which eliminates common database query errors during execution.

3.5 Real-time communication: MQTT and EMQX cloud

Real-time communication is crucial for distress broadcasting and rescuer coordination. The system utilizes MQTT [14] (Message Queuing Telemetry Transport) as the primary protocol for real-time location tracking and signal sharing. Unlike HTTP or standard WebSockets, MQTT has an extremely lightweight header size (minimizing bandwidth consumption) and operates on a Publish/Subscribe model. This is essential for FloodHelper, as victims often broadcast signals in areas with degraded mobile networks. MQTT's built-in Quality of Service (QoS) levels guarantee that critical SOS signals and location updates are retried and delivered once a connection is re-established.

The system uses EMQX cloud [15] as the managed MQTT broker to route messages between the mobile clients and the backend. EMQX cloud is selected over a self-hosted broker because it is built for distributed clustering, allowing the system to scale automatically and handle spikes in concurrent connections during active flood disasters without manual infrastructure administration.

On the server side, Python with the 'paho-mqtt' [16] library is used to run background processes that listen to incoming location streams. Python's event-driven, asynchronous client handles high-frequency location data from EMQX cloud without blocking the main NestJS API server, maintaining optimal server responsiveness.

3.6 Support services and infrastructure

3.6.1 Cloud storage: Cloudinary

Cloudinary [17] is integrated to manage dynamic media assets, such as images uploaded for charity verification campaigns or user profiles. In disaster zones, net-

work speed is highly limited. Cloudinary is chosen because it allows on-the-fly image optimization and resizing via URL parameters, reducing the file size delivered to mobile devices. Images are served through a global Content Delivery Network (CDN), offloading bandwidth from the primary backend server and ensuring fast image loading on the mobile client.

3.6.2 Push notifications: Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) [18] is employed to deliver push notifications, such as emergency alerts, rescue status updates, and announcement broadcasts. Since the mobile app must alert users even when it is running in the background or closed, FCM is selected as it is natively supported by both Android and iOS notification systems. It supports topic-based subscription, allowing the backend to broadcast notifications to all users in a specific ward simultaneously.

3.6.3 In-memory caching: Redis

Redis [19] is utilized as an in-memory database to store temporary, high-frequency data such as One-Time Passwords (OTPs) for phone authentication and user session states. Storing these short-lived tokens in Redis prevents unnecessary write operations to the primary PostgreSQL database. Redis's sub-millisecond read/write latency and automatic Time-To-Live (TTL) expiration ensure rapid, secure authentication workflows.

3.6.4 Containerization and hosting: Docker and Render

Docker [20] is used to containerize the NestJS backend and PostgreSQL database, packaging the applications with their exact environment configurations. This guarantees consistency between the developers' local environments and the production servers, eliminating deployment discrepancies.

Render [21] is selected as the cloud hosting provider for the backend services. Render is chosen over complex cloud providers (such as AWS or Google Cloud) because it provides a fully managed Platform-as-a-Service (PaaS) with integrated Git-based CI/CD. This automates the build and deployment process of the Docker containers whenever code is merged, reducing operational overhead and allowing the project to focus on feature implementation.

3.7 Development tools

Visual Studio Code [22] serves as the primary Integrated Development Environment (IDE) for writing, formatting, and debugging the codebase across both the frontend and backend services. It is chosen for its extensive extension ecosystem, which supports development in both Dart and TypeScript.

For version control and collaborative development, Git [23] and GitHub [24] are employed to manage repository history, track feature updates, and maintain code quality through pull request workflows. This setup ensures codebase consistency and enables seamless team collaboration.

Finally, Postman [25] is used to perform API testing, allowing developers to execute, validate, and document REST API requests and responses during development. It helps verify backend route correctness before integrating them into the frontend.

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

4.1 Architecture design

4.1.1 Software architecture selection

To meet the complex requirements and urgency of the FloodHelper system, the project adopts a combination of the Client-Server model and the Publisher-Subscriber architecture as the overall foundation. Theoretically, the Client-Server model strictly separates the roles of the client (interaction) and the server (centralized processing). Meanwhile, the Publisher-Subscriber pattern decouples message senders (Publishers) from receivers (Subscribers) via an intermediary (Broker), making it highly optimal for real-time data flows. Applied practically to FloodHelper, alongside standard HTTP communication, this architecture is actualized through the MQTT protocol. The mobile application acts as a Publisher, continuously broadcasting urgent data packets (such as location coordinate updates), while the Server and web management system act as Subscribers listening to specific topics. This ensures immediate processing and rescue map updates, maintaining continuity even in unstable network conditions.

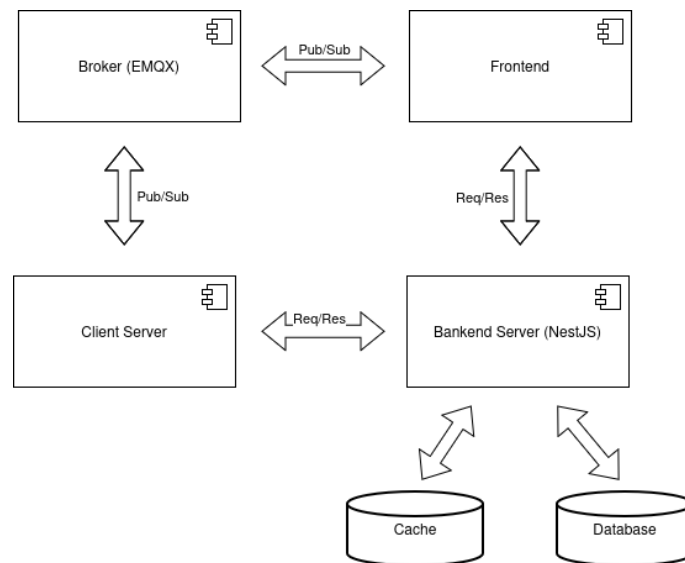


Figure 4.1: Software Architecture

Diving into the internal structure, the Client application moves beyond traditional MVVM by dividing the system into two distinct main layers: the ui layer and the data layer. The ui layer comprises Views and ViewModels (state management), while the data layer handles API interactions via Repositories, Services, DTOs, and Mappers. A significant architectural enhancement is the introduction of a domain buffer layer in between, acting as a framework-independent zone for data conver-

sion. This architecture strictly adheres to a unidirectional dependency rule: upper layers can only call lower layers, and upper layers react to lower-layer changes via an observation mechanism (watch). The entire initialization and Dependency Injection (DI) process on the Client is automated using the Riverpod package.

On the server side, a Layered Architecture is utilized to organize the source code into horizontal layers with distinct responsibilities, where the data flow moves unidirectionally downwards. This is realized through three components: Controller (receiving HTTP requests and routing), Service (handling core business logic), and Repository (executing database queries). To couple these layers loosely and adhere to the Inversion of Control (IoC) principle, the backend thoroughly leverages the built-in Dependency Injection mechanism of the NestJS framework. The consistent application of DI across both client and server ensures that components operate independently, enhancing maintainability and facilitating the automated testing process.

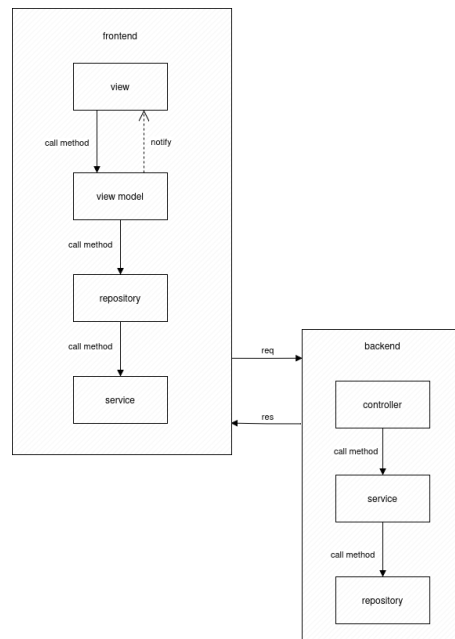


Figure 4.2: Layer Architecture for Frontend and Backend

4.1.2 Overall design

The overall architecture of the FloodHelper system is modeled through UML Package Diagrams, clearly demonstrating the separation of layers and strict dependency management principles. The packages are organized following a unidirectional dependency rule: the design ensures no circular dependencies, upper-layer packages depend on lower-layer packages, lower-layer packages strictly do not depend on upper ones, and the call flow does not skip layers. To provide a comprehensive view, the system design is divided into three diagrams: General Package

Diagram, Package Diagram for Frontend, and Package Diagram for Backend.

a, General package diagram

The general diagram outlines the highest-level structure of the entire system. This structure comprises three main blocks: *frontend*, *backend*, and *client_server*. Here, *frontend* and *client_server* act as clients, maintaining a one-way dependency on the *backend* for data exchange and retrieval. Internally, both the *frontend* and *backend* blocks are subdivided into smaller packages that adhere strictly to layered design principles.

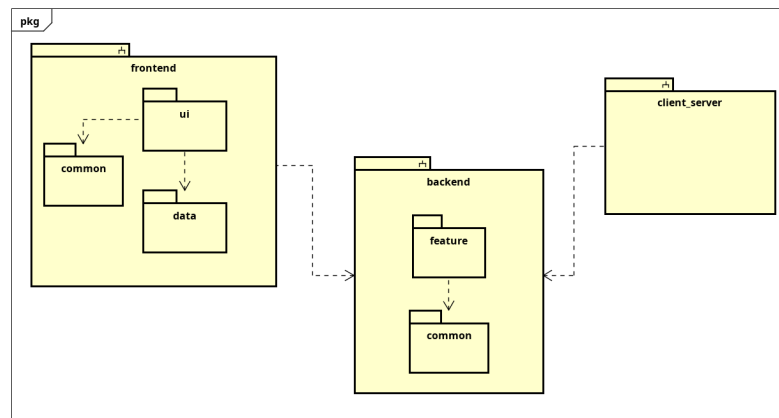


Figure 4.3: General package diagram

b, Package diagram for backend

The backend architecture is designed with two distinct layers: the *feature* layer (business logic) positioned at the top and the *common* layer (shared infrastructure) at the bottom. The *feature* layer depends on *common* for resources, while *common* operates entirely independently.

Package *feature*: Contains packages handling the core business logic of the application, including *charity* (charity campaign management categorized by *authority* and *user*), *admin* (system administration), *authentication* (user authentication), *profile* (profile management), *role_request* (handling permission requests), *signal* (transmitting and receiving distress signals), *announcement*, *address*, and *friend* (rescue contact management).

Package *common*: Provides shared tools and infrastructure connections utilized across the entire backend. This layer encompasses *prisma* (the ORM for database interactions), *vietqr* (payment/donation gateway integration), *firebase* (push notification services), and *cloud* (file storage services).

distress signaling and location tracking). This module spans all three architectural components: Frontend, Backend, and Client Server (Worker).

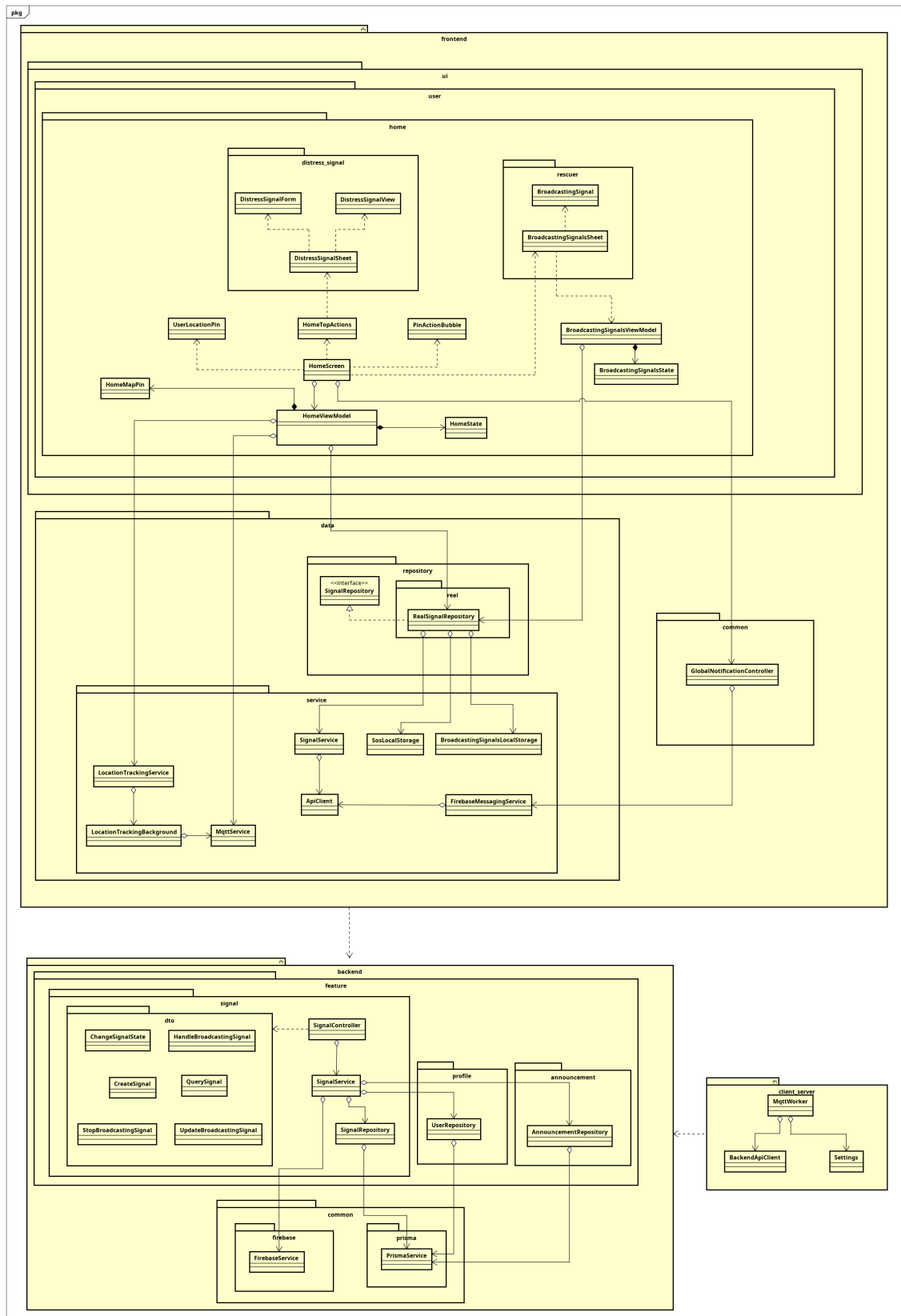


Figure 4.6: Detailed package design diagram for rescue module

a, Frontend subsystem

The design strictly adheres to the MVVM pattern and Clean Architecture, ensuring a unidirectional data flow.

Package *ui.user.home*: The core of the interface is the *HomeScreen* class. This class uses aggregation to integrate smaller UI components like *HomeMapPin*, *UserLocationPin*, *PinActionBubble*, and *HomeTopActions* to render the overview map. For user interactions, *HomeScreen* has a dependency on *DistressSignalSheet* (for signal creation) and *BroadcastingSignalsSheet* (for viewing broadcasting signal lists). Logically, *HomeScreen* depends on *HomeViewModel*. Following the design pattern, the ViewModels (*HomeViewModel* and *BroadcastingSignalsViewModel*) have a strict composition relationship with their respective state classes (*HomeState* and *BroadcastingSignalsState*).

Package *data*: To decouple the UI logic from data retrieval, the ViewModel only depends on the *SignalRepository* interface. The *RealSignalRepository* class provides the implementation of this interface. The *RealSignalRepository* maintains an aggregation relationship with underlying services such as *SignalService* (which calls APIs via *ApiClient*), *SosLocalStorage*, and *BroadcastingSignalsLocalStorage*.

To support real-time processing, the *HomeViewModel* depends directly on the *LocationTrackingService*. This service manages the background isolate/process (*LocationTrackingBackground*), which in turn depends on the *MqttService* to stream coordinates continuously via the MQTT protocol. Additionally, the *GlobalNotificationController* in the *common* package aggregates the *FirebaseMessagingService* to handle global push notifications.

b, Backend subsystem

On the backend, the Layered Architecture is clearly demonstrated through the separation of responsibilities.

The *SignalController* class is responsible for receiving requests and maintains a dependency on the *SignalService*.

The *SignalController* also depends on a set of Data Transfer Objects (*dto* package) such as *ChangeSignalState*, *CreateSignal*, etc., to validate and standardize inputs and outputs.

The *SignalService* encapsulates the business logic and depends on the *SignalRepository*. Finally, the *SignalRepository* depends on the *PrismaService* (located in the *common.prisma* package) to execute direct queries against the PostgreSQL

database.

c, Client server subsystem

To offload processing from the main backend API, the system employs an independent Worker.

The *MqttWorker* class has an aggregation relationship with *BackendApiClient* and *Settings*. This class acts as a subscriber, listening to coordinate data streams over MQTT, and depends on the backend infrastructure to update rescue data in real-time without disrupting the primary HTTP traffic flow.

4.2 Detailed design

4.2.1 User interface design

a, Target device specifications

The FloodHelper project is designed to serve two main target groups with two distinct platforms: a mobile app for citizens and a web app for Authorities and Admins. Detailed specifications for each platform are described in the table 4.1.

Table 4.1: Target device specifications

Feature	Mobile app	Web app
Target screen size	Supports sizes from 4.7 inches to 6.9 inches (Common aspect ratios: 16:9, 18:9, 19.5:9, and 20:9)	Laptop/Desktop screens from 13 inches up to 27 inches and above.
Screen resolution	Minimum: 720 x 1280 pixels (HD). Standard & Optimal: 1080 x 1920 pixels (FHD) up to 1440 x 3200 pixels (QHD+).	Minimum: 1024 x 768 pixels (Supports horizontal/vertical scrolling for older devices). Recommended & Optimal: 1366 x 768 pixels (Standard laptops), 1920 x 1080 pixels (Full HD) and above.

Continued on next page

Table 4.1 – Continued from previous page

Feature	Mobile app	Web app
Color support	Supports standard sRGB color gamut and DCI-P3 (for high-end devices) with 16 million colors (24-bit color depth), ensuring that warning icons and color-coded maps remain clearly visible even under bright outdoor lighting.	Supports True Color (24-bit) or 32-bit. High color contrast must be ensured to facilitate the distinction of terrain layers and flooded areas on satellite maps.
Touch & interaction	Supports Multi-touch. Optimizes tap target sizes to a minimum of 44x44 pt according to Apple and Google accessibility standards, ensuring users can interact effortlessly during panicky situations or when the screen is wet.	No
Input devices	No	Optimized for Mouse and Keyboard operations. Supports shortcuts to help dispatchers execute actions rapidly.

b, UI conventions

The user interface design system of FloodHelper is built upon three core criteria: clarity, speed, and safety. Regarding the color palette, the project utilizes dark blue as the primary color to convey reliability, combined with a light background for the web platform and dark mode for mobile to optimize battery life during power outages. A crucial emphasis is placed on the strictly regulated semantic color system: red is exclusively used for emergencies (SOS), orange for risk warnings, and green indicates safety or a successful status. To ensure optimal legibility across all devices, the system employs modern sans-serif fonts such as Inter or Roboto, featuring a clear size hierarchy ranging from large headings (24-32px) to caption text

(12px).

Interactive components, such as buttons and input fields, are designed with a transparent visual hierarchy to guide the user. Primary action buttons consistently stand out with a solid background color and rounded corners (8px on Web and 12-16px on Mobile for easy tapping), whereas secondary or text buttons feature a more minimalist design with outlines or no borders. Input fields and controls (checkboxes, radio buttons, toggles) are all mandated to have a minimum touch target size (e.g., 44px on Mobile) and display a blue border effect when focused. All user interactions, including pressing, hovering, or disabled states, provide synchronized visual feedback to simulate a realistic tactile feel.

In terms of spatial organization and navigation, the web interface utilizes a 12-column grid system with a left sidebar to accommodate a spacious administrative workspace, while the mobile platform optimizes a 4-column grid combined with bottom and right navigation bar for thumb-friendly reachability. The feedback messaging system is layered by priority: quick notifications (Toasts) appear at the bottom edge on mobile or the top corner on web, input errors are highlighted in red in real-time (Inline Validation), and critical decisions require a centralized modal dialog with a dimmed background overlay. Finally, all icons within the system adhere to a unified stroke style, with a strict mandatory rule: life-or-death functions like SOS or location sharing must always be accompanied by text labels and absolutely never stand alone to prevent any misinterpretation during moments of user panic.

c, Key features interface mockups

For the rescue feature, these mockups in figure 4.7 illustrate the primary home screen of the mobile application and its interactive state, featuring a comprehensive, full-screen map layout. The design utilizes a light theme to maximize the contrast of semantic color-coded status markers, where red pins with radiating rings denote active distress signals ("BROADCASTING") and green pins represent active rescue personnel or safe individuals. The base interface includes a bottom navigation bar, a top toggle control for filtering map data layers like "Transport" and "Weather," and a right-aligned vertical toolbar culminating in a prominent, solid blue "SOS" button for immediate emergency reporting. Upon specific user interaction, an "Active Distress Signals" Bottom Sheet slides up over a dimmed map overlay, maintaining spatial context while displaying a card-based list of ongoing emergencies. Each card prioritizes critical victim data—such as name, contact information, and essential survival metrics (number of trapped individuals, vulnerable demographics, and resource availability) displayed via intuitive pill-shaped

tags—alongside a solid dark blue "Locate" button to guide the rescuer's immediate response.

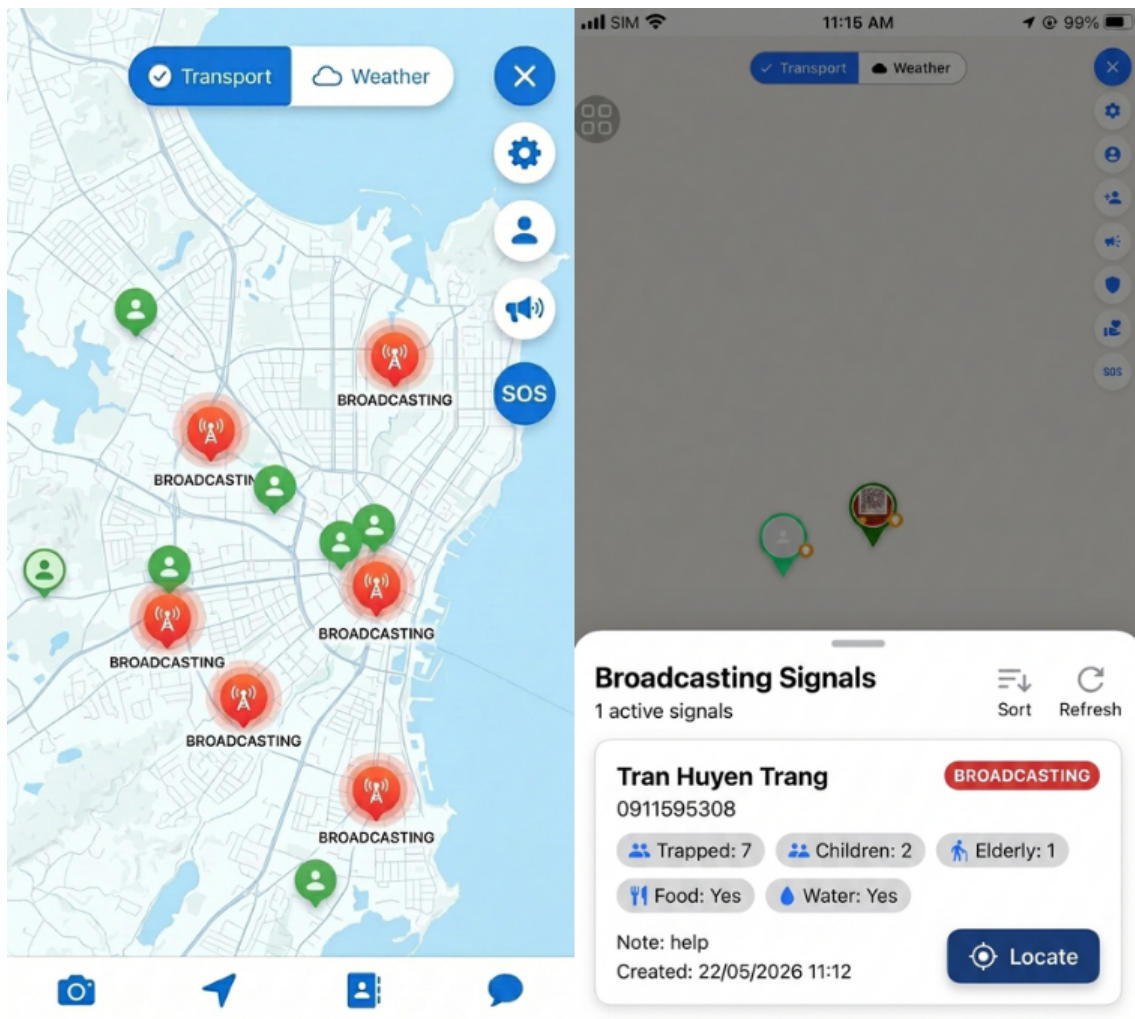


Figure 4.7: Mockup for Home Screen

For charity campaign feature, these mockups in figure 4.8 illustrate the user's charity campaign management flow. The left screen displays an overview list utilizing a horizontally scrollable tab bar (Created, Pending, Approved, Rejected, Done), enabling users to easily track and filter campaigns by their current stage. Campaigns are presented using a minimalist, card-based layout with a light gray background to visually separate them from the overarching white theme, clearly highlighting the campaign name and its creator. The right screen delves into the details of a specific campaign, providing a structured timeline of the donation and distribution phases. These specific dates are utilized by the system to execute the automated campaign state transition mechanism. In terms of interaction, the detailed interface employs a segmented control using a distinct orange color for the active "Allocation" tab to differentiate it from the inactive "Transaction" tab. Directly below is a standardized, prominent dark blue primary button for the "Post Announcement"

action. The remaining screen space is dedicated to a feed of operational updates and announcements, complete with image attachment support, thereby ensuring continuous information transparency throughout the project’s lifecycle.

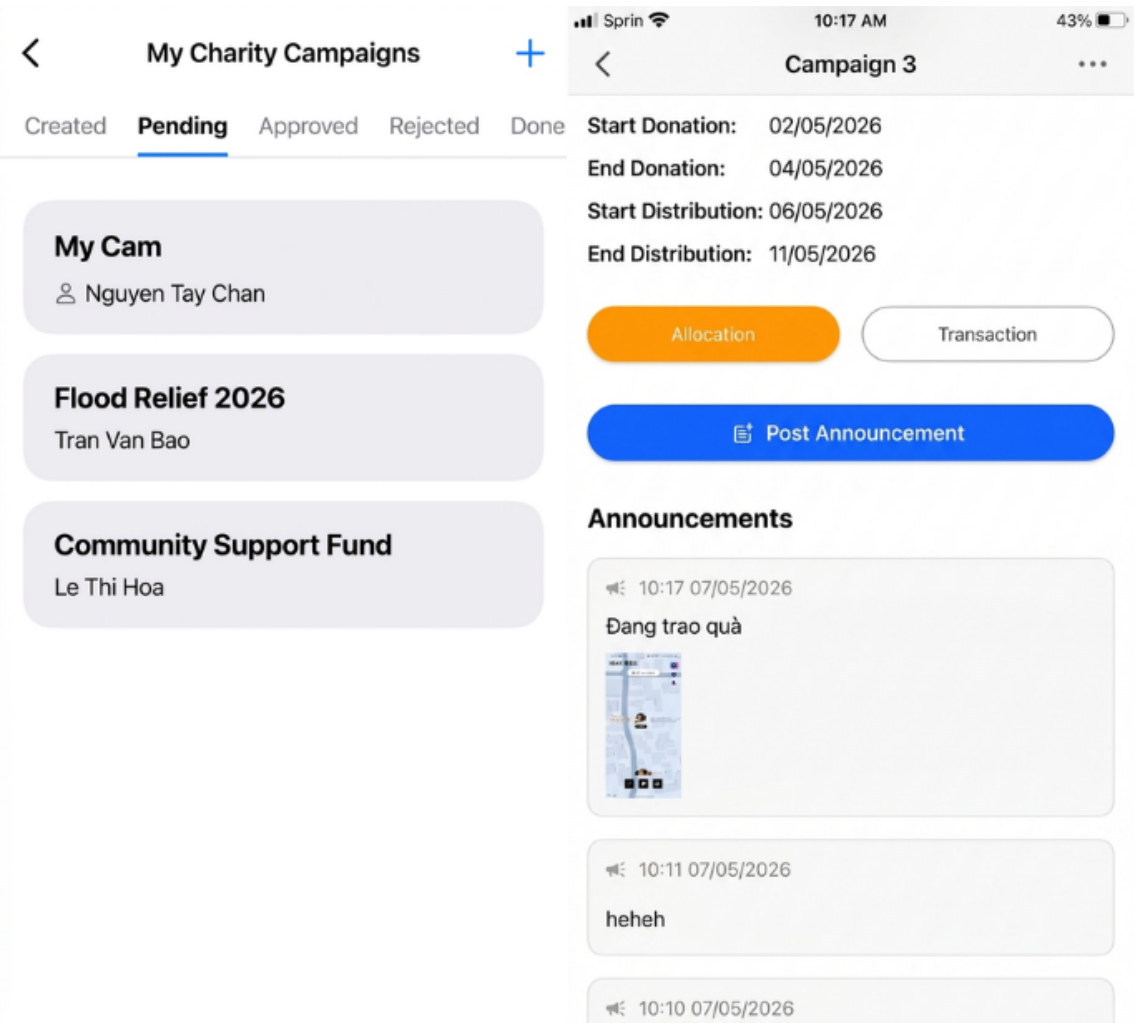


Figure 4.8: Mockup for Charity Campaign Management

For the web platform, the mockup in figure 4.9 illustrates the Web Dashboard interface exclusively designed for functional agencies and administrators (Authority Desk). The design strictly adheres to the established grid system and layout conventions, featuring a dark-themed Left Sidebar to create visual depth. This sidebar houses the primary functional modules and an expandable dropdown menu to filter charity campaigns by their specific states (ranging from Pending and Approved to Finished and Suspended). The Main Content area utilizes a clean white background and is structured using a Master-Detail layout to maximize the expansive display area of desktop screens. The left pane displays a chronological list of campaign requests, utilizing a blue-bordered card to distinctly highlight the currently active selection. The right pane presents the comprehensive details of the selected campaign, including its purpose, specific location, timeline milestones, and banking

information for disbursement. Notably, this design rigorously applies the Semantic Colors guidelines: the "APPROVED" status badge utilizes green (Success) to indicate authorization, while the secondary "Reject campaign" action is styled as an Outlined Button with a red border and text (Danger) to visually caution against the rejection action. Furthermore, the interface incorporates a prominently sized text area input for authorities to append a "Decision note" prior to executing any action, ensuring that the approval workflow remains fully transparent and traceable.

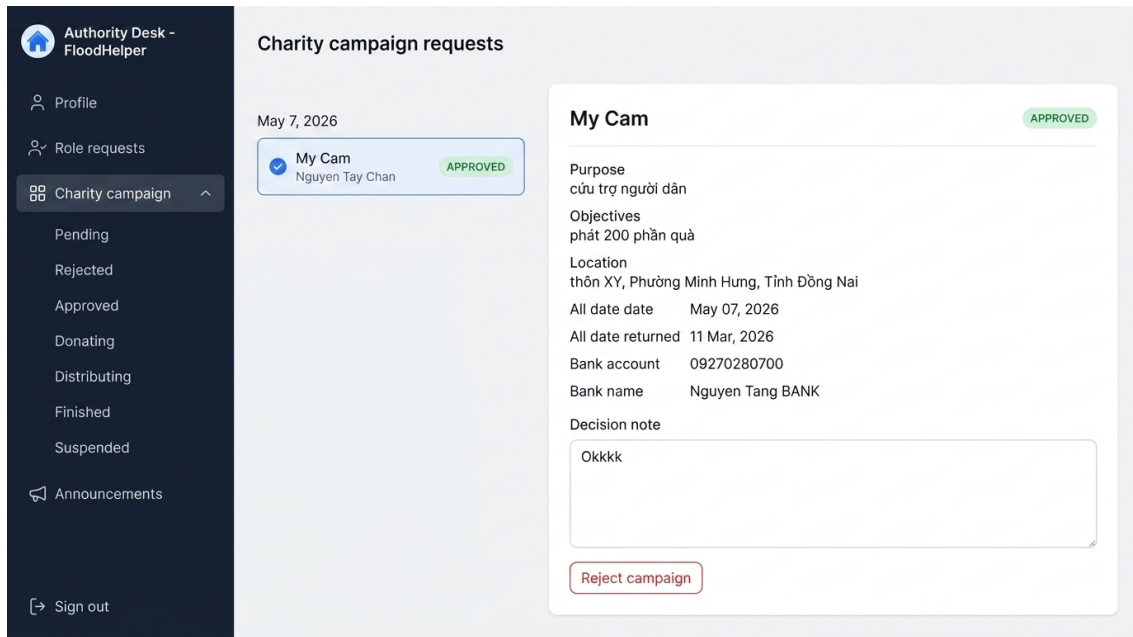


Figure 4.9: Mockup for Authority Screen

4.2.2 Layer design

The following tables are the detailed class specifications for three key classes in the rescue feature: SignalController, SignalService, and SignalRepository.

a, SignalController

Table 4.2: SignalController - Attribute Design

No	Name	Data type	Default	Description
1	signalService	SignalService	-	The injected service used to handle the business logic related to distress signals.

Table 4.3: SignalController - Operation Design

No	Name	Return type	Parameter	Description
1	create	Promise<any>	dto: CreateSignalDto	Receives a request from a user to create a new distress signal.
2	update-Broadcasting-InfoByUser	Promise<any>	dto: UpdateSignalInfoDto	Handles the request to update the information of an actively broadcasting distress signal by the user.
3	stop-Broadcasting-ByUser	Promise<any>	dto: StopBroadcastingByUserDto	Handles the request to stop broadcasting a distress signal by its creator.
4	handle-Broadcasting-ByUser	Promise<any>	dto: HandleBroadcastingByUserDto	Handles the request to mark a broadcasting distress signal as being handled by a rescuer.
5	listRescuer-Broadcasting	Promise<any>	-	Returns a list of actively broadcasting distress signals (for the Rescuer role).
6	listRescuer-Handled	Promise<any>	user: any	Retrieves a list of distress signals that have been handled by the current rescuer.
7	getMyLatest	Promise<any>	user: any	Retrieves the latest distress signal created by the current user.

b, SignalService

Table 4.4: SignalService - Attribute Design

No	Name	Data type	Default	Description
1	signal-Repository	Signal-Repository	-	Repository used to interact with the Signal table in the database.
2	user-Repository	User-Repository	-	Repository used to query user and rescuer information.
3	firebase-Service	Firestore-Service	-	Service used to send push notifications via Firebase.
4	announcement-Repository	Announcement-Repository	-	Repository used to create system announcements (Public Announcements).

Table 4.5: SignalService - Operation Design

No	Name	Return type	Parameter	Description
1	createSignal	Promise<any>	createdBy: string, dto: CreateSignalDto	Creates a distress signal, sends push notifications, and creates announcements for rescuers in the same ward.
2	updateInfo	Promise<any>	signalId: string, actorUserId: string, dto: UpdateSignalInfoDto	Updates the details of a broadcasting signal, ensuring ownership validation.

Continued on next page

Table 4.5 – Continued from previous page

No	Name	Return type	Parameter	Description
3	deleteSignal	Promise<any>	signalId: string, ac- torUserId: string	Deletes a distress signal from the system (restricted to the creator only).
4	changeState	Promise<any>	signalId: string, ac- torUserId: string, dto: ChangeSig- nalStateDto	Changes the signal state to either HANDLED or STOPPED .
5	listSignals	Promise<any>	query: QuerySig- nalsDto	Queries a list of signals based on createdBy or handledBy filters.
6	list- Broadcasting- Signals	Promise<any>	-	Returns a list of all distress signals currently in the BROADCASTING state.
7	listHandled- Signals- ByRescuer	Promise<any>	handledBy: string	Returns a list of signals that have been handled by a specific rescuer.
8	getLatest- SignalByUser	Promise<any>	createdBy: string	Returns the most recently created distress signal by a specific user.

Continued on next page

Table 4.5 – Continued from previous page

No	Name	Return type	Parameter	Description
9	update-Broadcasting-InfoByUser	Promise<any>	dto: UpdateSignalInfoDto	Finds the user's actively broadcasting signal and updates its information.
10	stop-Broadcasting-ByUser	Promise<any>	createdBy: string	Finds the user's actively broadcasting signal and updates its state to STOPPED.
11	handle-Broadcasting-ByRescuer	Promise<any>	createdBy: string, handledBy: string	Marks the signal as handled by a rescuer and sends push and system notifications to the victim.
12	ensureNo-Broadcasting-Signal	Promise<void>	createdBy: string	(Private) Ensures the user does not create a new signal if they already have one actively broadcasting.
13	getById-OrThrow	Promise<any>	signalId: string	(Private) Fetches a signal by ID or throws a NotFoundException if it does not exist.
14	is-Broadcasting-Unique-Violation	boolean	error: unknown	(Private) Checks for the Prisma P2002 error code to catch Unique Constraint violations.

c, SignalRepository

Table 4.6: SignalRepository - Attribute Design

No	Name	Data type	Default	Description
1	prisma	PrismaService	-	The Prisma client instance used to directly query the database.

Table 4.7: SignalRepository - Operation Design

No	Name	Return type	Parameter	Description
1	createSignal	Promise<any>	createdBy: string, data: any	Inserts a new signal record into the database with a default state of BROADCASTING.
2	getSignal	Promise<any>	signalId: string	Retrieves a specific signal record along with information about its creator and the handler.
3	listActive-Signals	Promise<any>	-	Retrieves a list of all signal records currently in the BROADCASTING state.
4	listSignals-ByUser	Promise<any>	userId: string	Queries all distress signals created by a specific user.
5	listSignals	Promise<any>	where: Prisma.Sig- nalWhereInput	Searches for signals based on a customized Prisma where condition object.

Continued on next page

Table 4.7 – Continued from previous page

No	Name	Return type	Parameter	Description
6	findActive-SignalByUser	Promise<any>	createdBy: string	Finds the single active (BROADCASTING) signal created by a user.
7	getLatest-SignalByUser	Promise<any>	createdBy: string	Returns the most recently created signal record for a specific user.
8	listHandled-SignalsBy-Rescuer	Promise<any>	handledBy: string	Retrieves a list of signal records in the HANDLED state that were processed by a specific rescuer.
9	updateSignal-State	Promise<any>	signalId: string, state: string, handledBy?: string	Updates a signal record's state and records the timestamp and handler information if applicable.
10	stopSignal	Promise<any>	signalId: string	Updates a signal's state to STOPPED and logs the timestamp of when it was stopped.
11	deleteSignal	Promise<any>	signalId: string	Deletes a distress signal record from the database.

Continued on next page

Table 4.7 – Continued from previous page

No	Name	Return type	Parameter	Description
12	findById	Promise<any>	id: string	Overrides the Base method: Calls the getSignal method internally.
13	findAll	Promise<any>	-	Overrides the Base method: Calls the listActiveSignals method internally.
14	create	Promise<any>	data: any	Overrides the Base method: Throws an error instructing the use of the specialized createSignal method instead.
15	update	Promise<any>	id: string, data: any	Overrides the Base method: Arbitrarily updates a record's data based on its ID.
16	delete	Promise<any>	id: string	Overrides the Base method: Calls the deleteSignal method internally.
17	count	Promise-<number>	-	Returns the total count of distress signal records currently in the database.

d, Sequence Diagram

To further illustrate the dynamic interactions between the designed classes, the following sequence diagrams detail the execution flow of two core use cases in server side: Broadcast Signal and Handle Signal. These diagrams map out the exact

sequence of method calls across the Controller, Service, and Repository layers.

The sequence diagram in figure 4.10 demonstrates the execution flow when a user initiates a new distress signal. The process begins with the `SignalController` receiving the creation request and delegating it to the `SignalService`. First, the service queries the `SignalRepository` to ensure the user does not already have an actively broadcasting signal. Once validated, the system persists the new signal entity in the database. To alert the appropriate personnel, the service retrieves the creator's profile via the `UserRepository` to identify their specific ward and subsequently fetches a list of available rescuers within that area. Finally, the `FirebaseService` dispatches a multicast push notification to these designated rescuers, while the system iteratively logs a formal system announcement for each rescuer through the `AnnouncementRepository`.

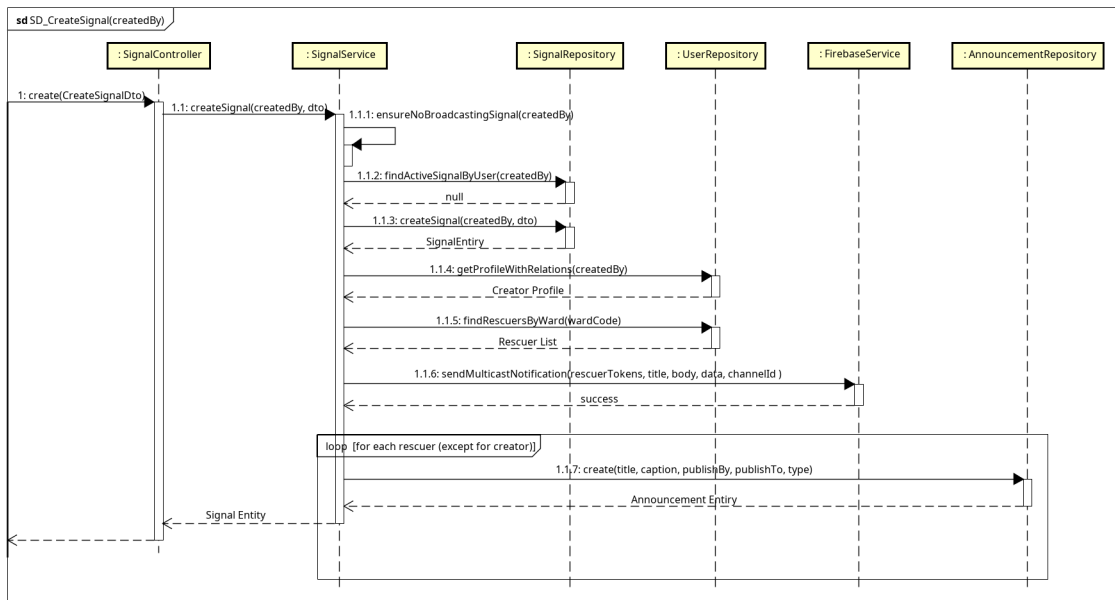


Figure 4.10: Sequence diagram for use case "Broadcast signal" on the server side

Next, this sequence diagram in figure 4.11 illustrates the backend operations triggered when a rescuer acknowledges and commits to handling a victim's active signal. Initially, the `SignalController` invokes the state update process within the `SignalService`. The service then uses the `SignalRepository` to locate the victim's active signal and immediately updates its status to "HANDLED", linking it directly to the rescuer's ID. Following this database update, the system retrieves the comprehensive profile information of both the handler (rescuer) and the creator (victim) through the `UserRepository`. Utilizing the victim's retrieved FCM token, the `FirebaseService` sends a direct push notification to inform the victim that assistance is underway. The transaction is completed when

the `AnnouncementRepository` logs an official system announcement to permanently record the handling event.

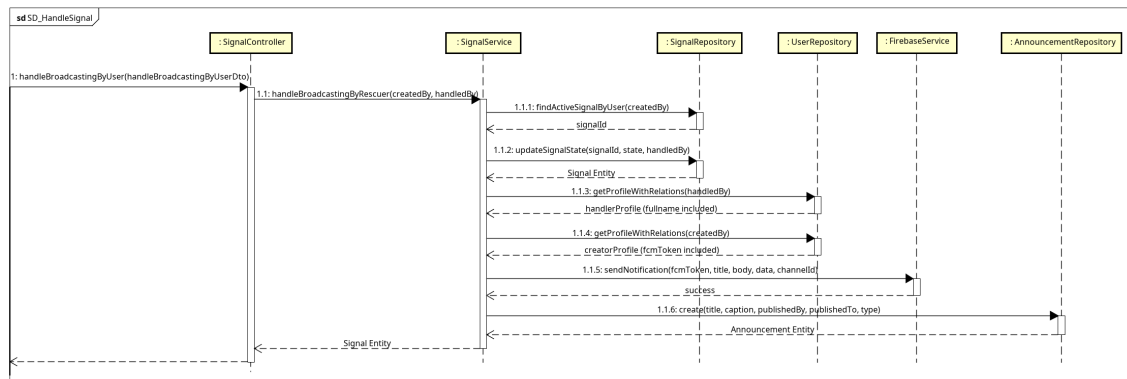


Figure 4.11: Sequence diagram for use case "Handle signal" on the server side

4.2.3 Database design

The FloodHelper database is designed to support the system's core functionalities, including user management, social networking, emergency rescue coordination, charity campaign management, and real-time communication. The database follows a relational model implemented using PostgreSQL, providing a structured and consistent mechanism for storing application data.

As illustrated in Figure 4.12, the database schema is organized into several functional modules. The user management module is centered around the User entity, which stores user information and is associated with entities such as Account, Profile, Session, and Provider to support authentication, profile management, and session tracking. Social networking functionalities are implemented through entities including Friendship, FriendMakingRequest, Post, Comment, and Like, enabling users to establish relationships and interact with shared content.

Figure 4.12: Entity-relationship diagram

Emergency rescue operations are supported by the Signal entity, which stores distress requests submitted by victims and the associated rescue information. In addition, communication between users is facilitated through ChatRoom, RoomMember, and Message, while geographical reference data are maintained using the Province and Ward entities. The WeatherMap entity stores weather information that supports rescue-related functionalities.

cations. Figure 4.13 presents the resulting relational database schema implemented in PostgreSQL.

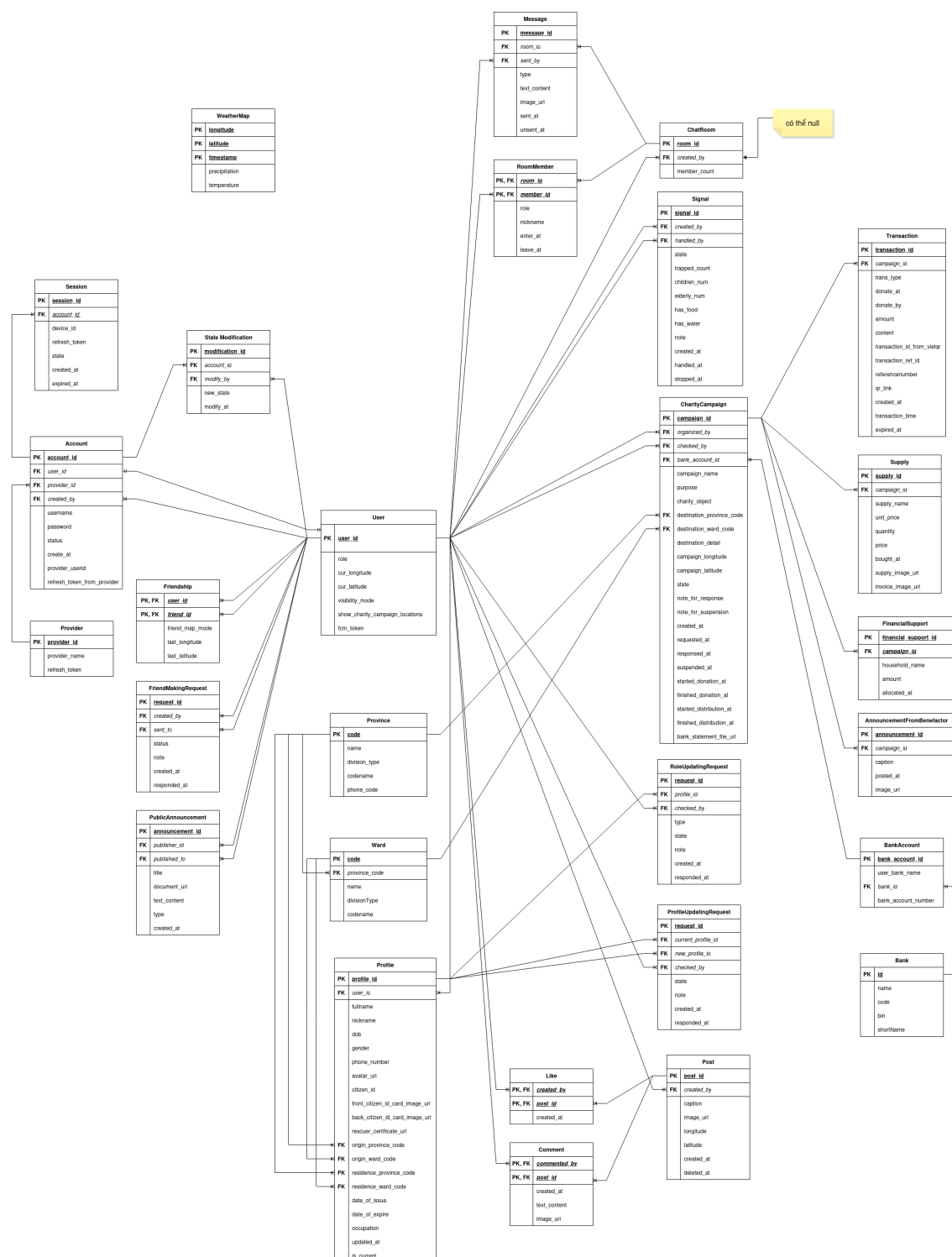


Figure 4.13: Database schema

4.3 Application building

4.3.1 Libraries and tools

Table 4.8 lists the programming languages, software development kits, key libraries, APIs, and development tools utilized to build the various modules of the FloodHelper system, along with their respective versions and official resources.

Table 4.8: List of languages, libraries, APIs, and tools used in the entire system

Module / Purpose	Tool / Library	Version	URL
1. Frontend module			
Programming language	lan- Dart SDK	$\geq 3.10.1$ $< 4.0.0$	https://dart.dev/
Development work	frame- Flutter SDK	$\geq 3.38.1$	https://flutter.dev/
State management	flutter_riverpod	2.6.1	https://pub.dev/packages/flutter_riverpod
HTTP API Client	dio	5.9.1	https://pub.dev/packages/dio
Digital map display	flutter_map	8.2.2	https://pub.dev/packages/flutter_map
Get GPS coordinates	geolocator	14.0.2	https://pub.dev/packages/geolocator
Distance calculation	latlong2	0.9.1	https://pub.dev/packages/latlong2
Cloud push notifications	firebase_messaging	15.2.10	https://pub.dev/packages/firebase_messaging

Continued on next page...

Table 4.8 (continued)

Module / Purpose	Tool / Library	Version	URL
Firebase core library	firebase_core	3.15.2	https://pub.dev/packages/firebase_core
MQTT connection & messaging	mqtt_client	10.11.9	https://pub.dev/packages/mqtt_client
Screen routing (Navigation)	go_router	14.8.1	https://pub.dev/packages/go_router
Background service	flutter_background_service	5.1.0	https://pub.dev/packages/flutter_background_service
Local push notifications	flutter_local_notifications	21.0.0	https://pub.dev/packages/flutter_local_notifications
Local configuration storage	shared_preferences	2.2.3	https://pub.dev/packages/shared_preferences
Localization & Formatting	intl	0.20.2	https://pub.dev/packages/intl
Image picker (gallery/camera)	image_picker	1.2.1	https://pub.dev/packages/image_picker

Continued on next page...

Table 4.8 (continued)

Module / Purpose	Tool / Library	Version	URL
Device file picker	file_picker	8.3.7	https://pub.dev/packages/file_picker
Image display & zoom	photo_view	0.14.0	https://pub.dev/packages/photo_view
SVG vector image display	flutter_svg	2.2.3	https://pub.dev/packages/flutter_svg
2. Backend module			
Programming language	lan- TypeScript	5.9.3	https://www.typescriptlang.org/
Server runtime environment	Node.js	>= 20.x	https://nodejs.org/
Core development framework	NestJS (@nestjs/core)	11.1.9	https://nestjs.com/
Database ORM	Prisma (@prisma/client)	5.22.0	https://www.prisma.io/
PostgreSQL connection library	pg (node-postgres)	8.16.3	https://node-postgres.com/
Firebase Admin (FCM Sending)	firebase-admin	13.7.0	https://firebase.google.com/
Cloud image upload & storage	cloudinary	2.10.0	https://cloudinary.com/
Quick QR payment integration	vietqr	1.1.9	https://vietqr.io/
System email sender	nodemailer	7.0.12	https://nodemailer.com/

Continued on next page...

Table 4.8 (continued)

Module / Purpose	Tool / Library	Version	URL
NestJS mailer module	@nestjs-modules/mailer	2.0.2	https://github.com/nest-modules/mailer
Security & password encryption	bcrypt	5.1.1	https://github.com/kelektiv/node.bcrypt.js
JWT authentication	passport-jwt	4.0.1	https://www.passportjs.org/
Google account authentication	passport-google-oauth20	2.0.0	https://www.passportjs.org/
Cache management	cache-manager	7.2.8	https://github.com/jakec/node-cache-manager
Redis support	cache-manager-redis-store	3.0.1	https://github.com/dabroek/node-cache-manager-redis-store
DTO validation	class-validator	0.14.3	https://github.com/types/tack/class-validator
Environment variable loader	dotenv	17.2.3	https://github.com/motdootla/dotenv
3. MqttClient module			
Programming language	lan- Python	3.x	https://www.python.org/

Continued on next page...

Table 4.8 (continued)

Module / Purpose	Tool / Library	Version	URL
API Client building	FastAPI	>= 0.100.0	https://fastapi.tiangolo.com/
API execution server (ASGI)	Uvicorn	>= 0.23.0	https://www.uvicorn.org/
MQTT Broker connection	paho-mqtt	>= 1.6.1	https://pypi.org/project/paho-mqtt/
HTTP communication to Backend	requests	>= 2.31.0	https://requests.readthedocs.io/
4. IDE, testing, code standards & deployment			
Programming IDE	Visual Code	Studio Latest	https://code.visualstudio.com/
Source code version control	Git	Latest	https://git-scm.com/
Virtualization environment	Docker	Latest	https://www.docker.com/
Backend testing tool	Jest	30.2.0	https://jestjs.io/
Code quality linter	ESLint	9.18.0	https://eslint.org/
Source code formatter	Prettier	3.4.2	https://prettier.io/

4.3.2 Achievement

The primary achievement of the FloodHelper project is the successful design, implementation, and packaging of a comprehensive, multi-platform software ecosystem designed to facilitate disaster response and charity coordination activities. The system has been fully built, packaged, and verified through local deployments, comprising three distinct packaged products operating concurrently on a unified

relational database.

a, Packaged software products and component details

This section describes the delivery package formats, key components, and architectural roles of each product within the FloodHelper ecosystem. For clarity, these details are summarized in Table 4.9.

Table 4.9: Details of packaged software products

Product / Module	Delivery package	Key components	Significance and role
Frontend client (Mobile app & Web admin desk)	Compiled Android APK (app-debug.apk: 106.49 MB, app-release.apk: ≈ 22 MB) and compiled static HTML/JS assets.	Flutter layout widgets (HomeScreen map dashboard, DistressSignalSheet, Charity transaction cards, authority views), state providers (Riverpod), and client communication integrations (Dio, Geolocator, MqttClient).	Serves as the interactive touchpoint. The mobile app lets users stream real-time coordinate changes via MQTT and dispatch SOS signals. The web app acts as the dispatch dashboard for rescue organizations to coordinate operations and approve charity requests.

Continued on next page

Table 4.9 – Continued from previous page

Product / Module	Delivery package	Key components	Significance and role
Backend server (NestJS app)	Self-contained Docker Image (floodhelper-backend:latest $\approx$ 320 MB) containing NestJS compiled JavaScript in dist/ folder (1.60 MB, 430 files).	REST Controllers mapping API routes, NestJS services for SOS validation and auth filters, Prisma database clients, and third-party integrations (Firebase Admin SDK, VietQR integration, Cloudinary API).	Acts as the central business engine of the system, verifying accounts, securing data access boundaries, checking DTO inputs, managing notification triggers, and handling transactional email dispatches.

Continued on next page

Table 4.9 – Continued from previous page

Product / Module	Delivery package	Key components	Significance and role
MQTT worker (Data coordinator)	Lightweight Docker Image (floodhelper-mqtt-worker:latest $\approx$ 120 MB) running Python 3 and FastAPI.	Persistent Paho-MQTT connection client, temporary background message queue handlers, and API request forwarding dispatcher.	Acts as a real-time data coordinator. It subscribes to high-frequency MQTT coordinate streams from active devices. Instead of writing directly to the database, it processes and forwards coordinate payloads to main NestJS server REST endpoints to offload HTTP connection traffic and preserve architecture layers.

Continued on next page

Table 4.9 – Continued from previous page

Product / Module	Delivery package	Key components	Significance and role
Database schema (PostgreSQL)	Database initialization scripts and migration files generated via <code>prisma-schema</code> . <code>prisma</code> .	Structured tables (User, Account, Profile, Signal, Friendship, Charity-Campaign, Transaction, geocoded tables for Province and Ward).	Provides persistent storage with transactional integrity, enforcing one-to-one mapping for accounts, one-to-many associations for tracking history, and multi-relational structures for volunteer messaging and spatial tracking.

b, Source code and package metrics

To evaluate the engineering scope of the system, a static code analysis was conducted across all active repositories. Excluded from this assessment are third-party libraries (`node_modules`, generated Flutter assets in `.dart_tool`, test cache files, and report documents). Table 4.10 details the breakdown of source files, package modules, class counts, source lines of code (SLOC), and physical sizes.

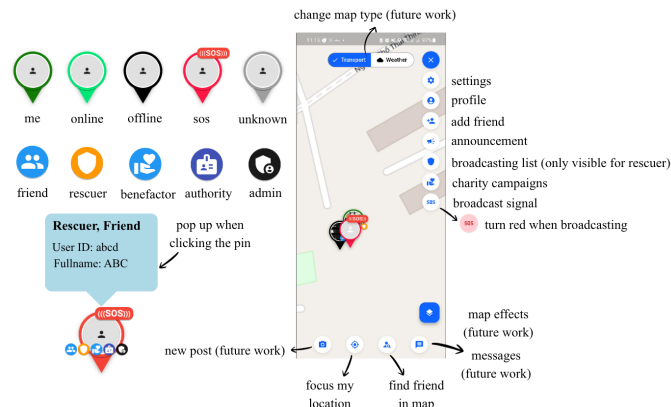
Table 4.10: Summary of source code and packaged product statistics

Module / Product	Language	Files	Packages	Classes	Lines of Code*	Packaged Product & Size
Frontend Client (Mobile & Web)	Dart	279	81	409	41,667 (36,311 code)	app-debug.apk (106.49 MB) / Release APK ($\approx$ 22 MB)
Backend Server (API Application)	TypeScript	145	36	123	13,299 (11,106 code)	Docker Container ($\approx$ 320 MB) / Dist folder (1.60 MB)
MQTT Client (Real-time Worker)	Python	6	2	3	455 (364 code)	Docker Container ($\approx$ 120 MB) / Source code (15.39 KB)
Total System	Dart/TS/Py	430	119	535	55,421 (47,781)	Full software suite

*Note: Lines of code are shown as Total Lines (Source Lines of Code excluding blanks and comments in parentheses).

4.3.3 Illustration of main functions

This section highlights the core interfaces of the application, detailing the most critical and interactive features designed for the users. Please refer to the annotated figure 4.14 for a detailed visual breakdown of these elements.

**Figure 4.14:** Home screen

The central component of the user experience is the Home screen, an interactive, real-time map dashboard that appears immediately upon successful login. This interface utilizes a color-coded pin system to represent real-time user statuses, allowing users to easily identify their own location with a green pin, alongside others who are online (cyan), offline (black), or unknown (grey). Crucially, users actively broadcasting an emergency are marked with a distinct red "SOS" pin. To further facilitate quick decision-making, these map pins are accompanied by specific icons denoting user roles, including Friend, Rescuer, Benefactor, Authority, and Admin. By tapping on any user's pin, an interactive pop-up window emerges, displaying essential identification details such as the selected user's assigned roles, User ID, and full name.

Navigating the application is streamlined through a floating, vertical menu on the right side of the screen. This panel provides one-tap access to general features like account settings, user profiles, friend management, system announcements, and charity campaigns. It also serves as the hub for emergency tools, featuring a dedicated SOS broadcast signal button that visibly turns red during an active transmission, as well as a broadcasting list exclusively accessible to users holding the 'Rescuer' role. Finally, the interface provides essential mapping controls at the bottom, allowing users to quickly focus on their current location or find friends on the map. The screen also strategically integrates placeholders for planned future developments, including options to create new posts, access messages, toggle map types (such as transport or weather overlays), and apply various map effects.

Following the SOS broadcast feature introduced on the Home screen, the application provides tailored interfaces for both victims and rescuers to manage emergencies efficiently. Looking at the figure 4.15, we can observe the distress signal process specifically designed from the victim's perspective. Before broadcasting, the user is presented with a comprehensive "Distress Signal" form. Here, they are prompted to input critical details about their current situation, such as the total number of trapped individuals, the specific count of vulnerable people like children and the elderly, and the availability of essential resources including food and water. An optional text field is also provided for any additional crucial information. Once the user clicks the prominent red "Broadcast Signal" button, the interface seamlessly transitions to its active state. After broadcasting, the screen displays a summary of the submitted data under an "Active Distress Signal" warning, clearly marked with a red "BROADCASTING" tag. From this active view, the victim retains control over their alert and can choose to either "Edit" the information if their situation changes or completely "Revoke Signal" once they are safe and no longer

require assistance.

For victim

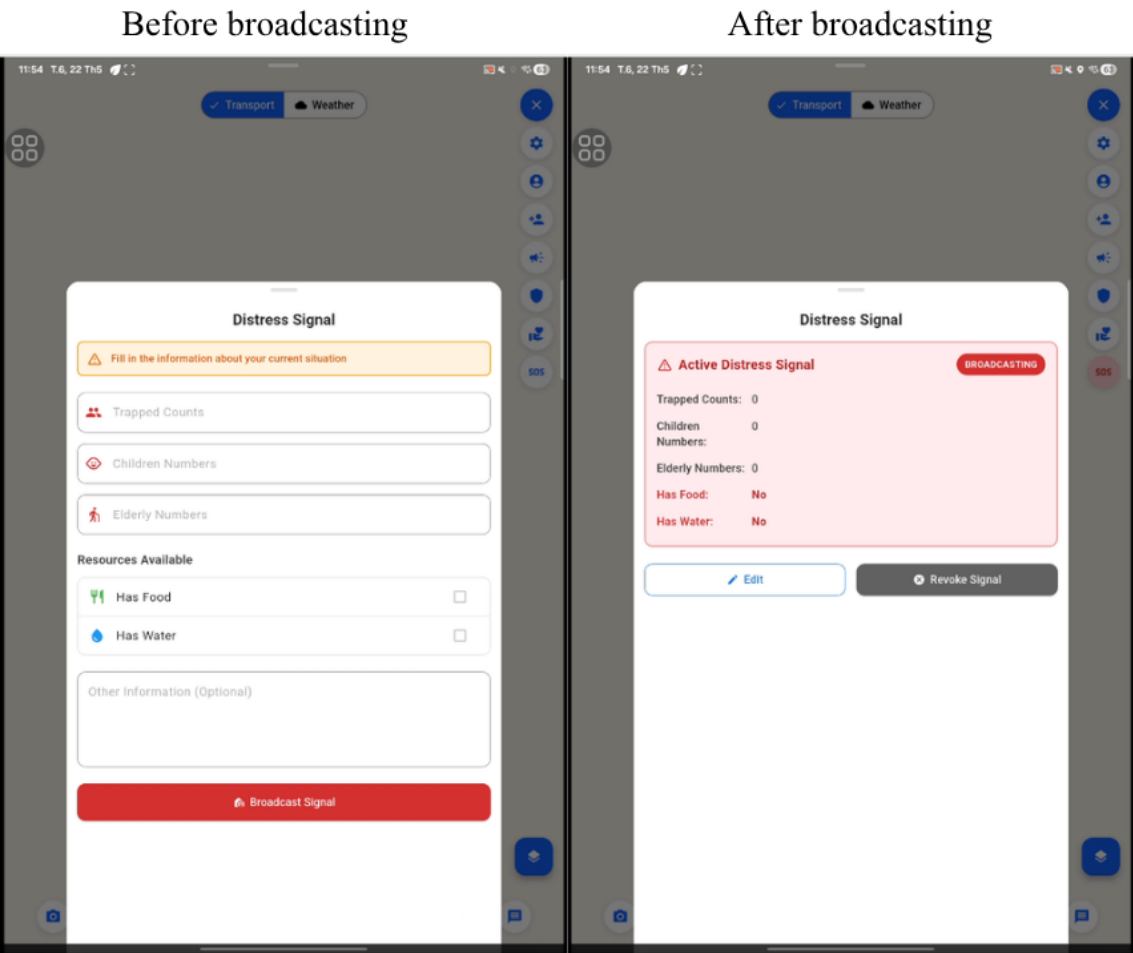
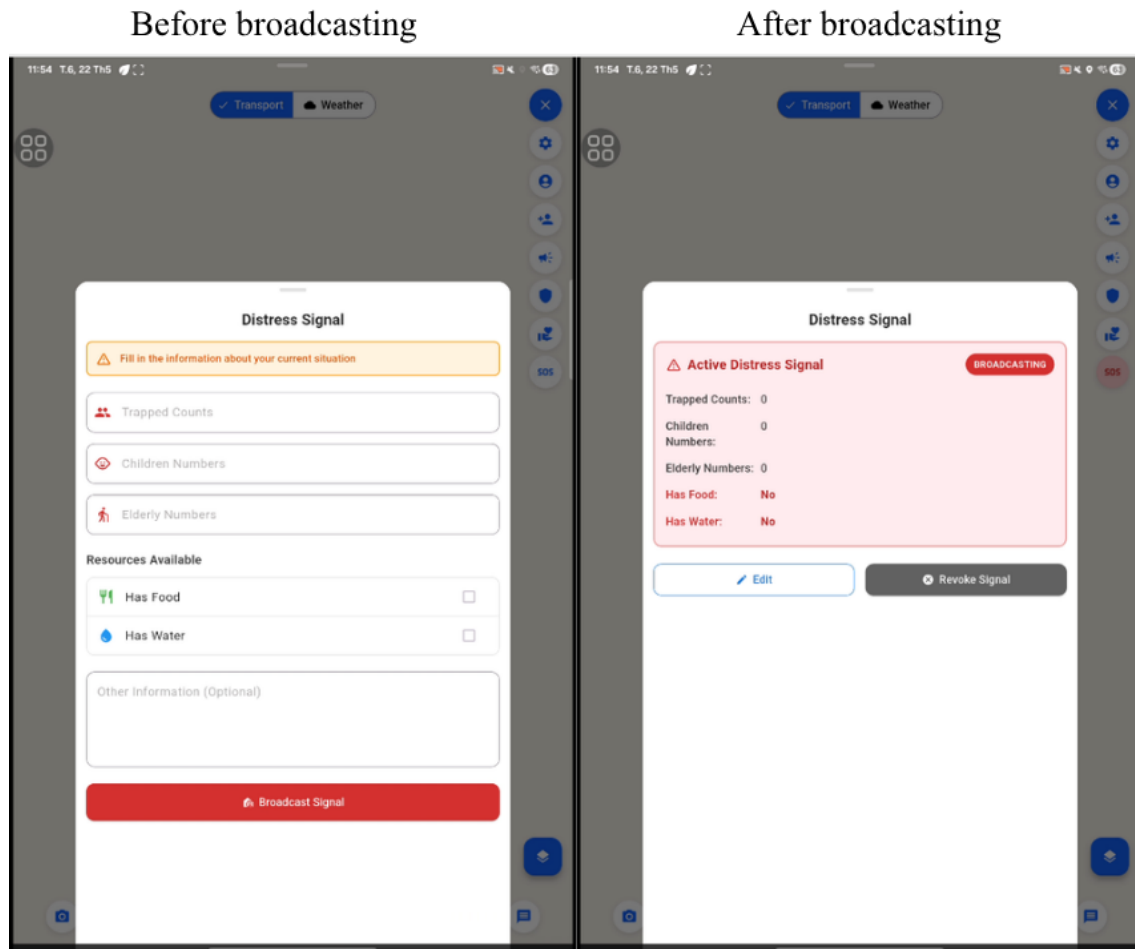


Figure 4.15: Broadcast signal for user

Conversely, the system equips authorized responders with specialized tools to track and manage these incoming emergencies. As illustrated in the figure 4.16, users holding the Rescuer role have access to a dedicated "Broadcasting Signals" interface. This screen acts as a centralized dashboard displaying a sortable list of all currently active distress signals in the network. Each active emergency is represented by a detailed card that provides the rescuer with immediate, vital context without needing to open a separate page. This information includes the victim's name, contact phone number, the time the signal was created, and a quick-read breakdown of their specific situation parameters, such as the exact counts of trapped individuals and resource scarcity. Most importantly, as highlighted by the annotation in the figure, each emergency card features a dedicated "Locate" button. Tapping this button directly focuses the main map on that specific victim's coordinates, allowing rescuers to swiftly pinpoint their exact location and coordinate a physical response.

For victim

**Figure 4.16:** Signal list for rescuer

Following the critical emergency response features, the application also facilitates long-term community support through its dedicated charity campaign system. Looking at the figure 4.17, we can observe the general "Charity Campaign" interface that is accessible to the entire user base. This screen provides a centralized feed of ongoing relief efforts, which are neatly organized into intuitive tabs based on their current operational phase, such as "Donating," "Distributing," and "Finished." Each campaign card displays the campaign's title along with the organizer's name, allowing the community to easily browse and follow various initiatives. Furthermore, as highlighted by the annotation in the figure, this public-facing screen includes a specific "My Campaigns" link positioned at the top right corner. This link serves as a secure gateway and is exclusively visible to users who hold the 'Benefactor' role, granting them access to their personalized management dashboard.

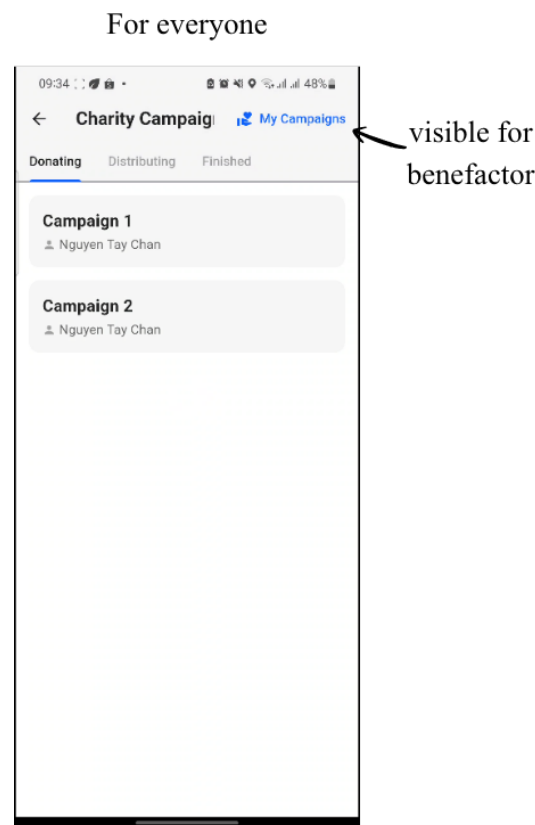


Figure 4.17: View campaigns for everyone

Transitioning to the administrative side of these relief efforts, the figure 4.18 illustrates the specialized tools provided to campaign organizers. The left side of this figure shows the "My Charity Campaigns" view, where benefactors can track their specific initiatives across a more granular set of statuses, including suspended operations. A prominent plus ("+") icon at the top of the screen allows these users to seamlessly create a new campaign. When a benefactor opens a specific initiative, as seen on the right side of the figure, a comprehensive detail panel emerges. This panel consolidates crucial logistical and financial transparency data, displaying the organizer's name, designated bank account details, the campaign's core purpose, and strict timelines for both the donation and distribution phases. Notably, next to the "Relief Location" text, there is a dedicated map icon that allows organizers to quickly "check in location" on the main map. Below these details, the interface equips benefactors with essential management controls, including buttons to handle allocations and transactions, as well as a "Post Announcement" feature to publish real-time updates directly to the campaign's timeline.

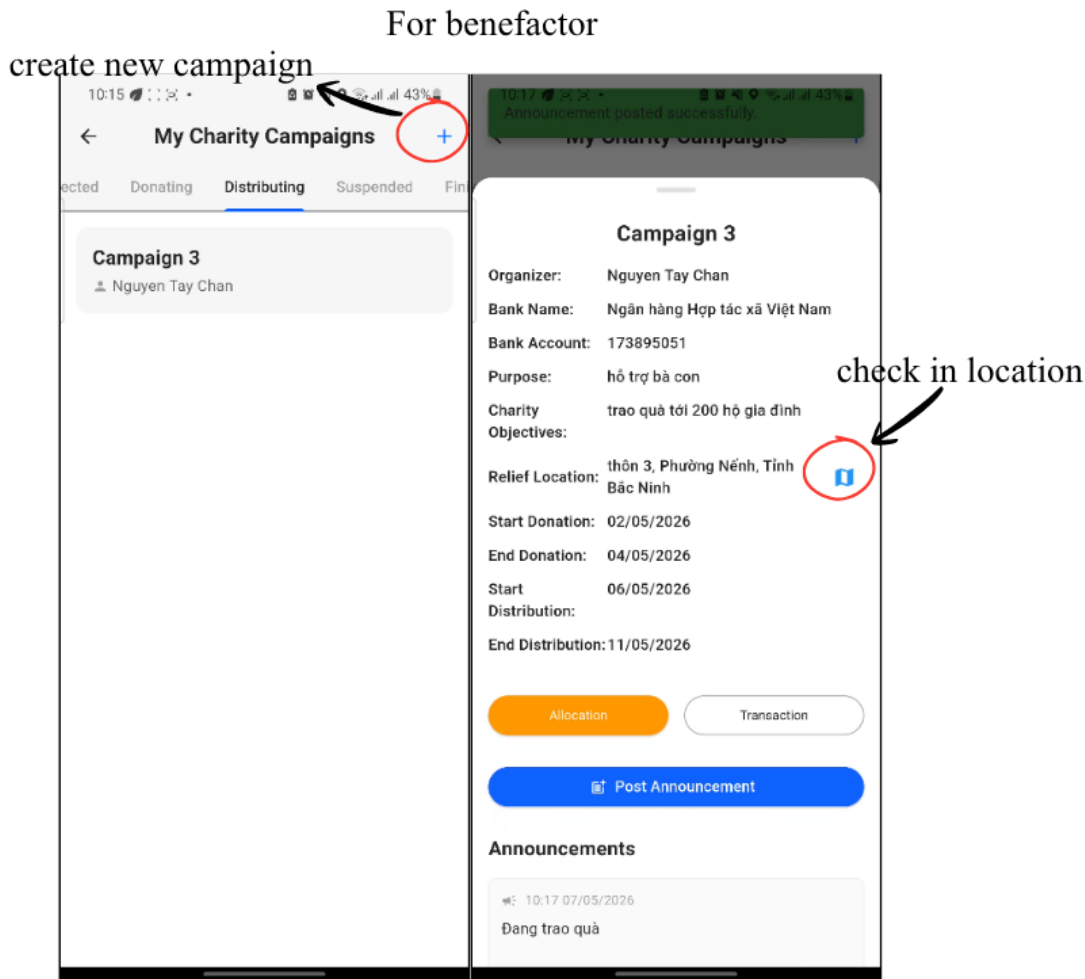


Figure 4.18: Campaign management for benefactor

4.4 Testing

To verify that the FloodHelper ecosystem operates reliably under field conditions, a multi-layered testing strategy was implemented. The testing workflow covers individual components, data pipelines, real-time protocols, and interactive user journeys.

4.4.1 Testing methodology and techniques

The system quality assurance process was conducted across three distinct testing levels:

- **Unit testing:** Implemented on the NestJS backend (using Jest) to test services, and on the Flutter frontend (using Flutter Test) to verify Riverpod view models and business validation logic.
- **Integration and E2E testing:** Focused on data transfers between the mobile client, the Python MQTT Worker, the NestJS server, and the database. Payment callbacks (VietQR hook) were also validated.

- **UI and widget testing:** Performed UI verification using Flutter’s widget testing framework. This includes golden image rendering tests to ensure map pins and role badges render consistently across varied resolutions.

The test cases were designed using the following techniques:

- **State transition testing:** Used to verify status transitions for SOS signals (BROADCASTING → HANDLED → STOPPED) and charity campaigns (Pending → Approved/Rejected → Finished) without deadlock.
- **Equivalence partitioning (EP) and boundary value analysis (BVA):** Utilized to define validation boundaries for geographic coordinates, integer limits (such as trapped victims count), and payment limits.
- **Use case testing:** Developed based on user stories representing critical field actions to verify collaboration across multiple systems.

4.4.2 Test case design for core functions

Testing was focused on three core functionalities: Distress Signaling, Real-time Location Tracking, and Charity Campaign Management. Representative test cases highlighting these core features are summarized in Table 4.11.

Table 4.11: Representative core test cases summary

Test ID	Feature	Key action & expected outcome	Status
TC-SOS-001	Distress Signal	Victim submits SOS request; status transitions to BROADCASTING and alerts are dispatched.	Pass
TC-SOS-003	Distress Signal	Rescuer accepts signal; status transitions to HANDLED and victim receives push notification.	Pass
TC-GPS-001	GPS Tracking	App publishes coordinate changes; MQTT worker parses and updates backend in real-time.	Pass
TC-GPS-003	GPS Tracking	Network drops; updates buffer locally and sync to database chronologically upon recovery.	Pass

Continued on next page

Table 4.11 – Continued from previous page

Test ID	Feature	Key action & expected outcome	Status
TC-CHA-002	Charity Campaign	Admin approves pending campaign; campaign is activated and QR code generated via VietQR.	Pass
TC-CHA-003	Charity Campaign	Bank payment callback matches transaction ID; system automatically updates campaign funds.	Pass

4.4.3 Summary of test results

A total of 85 automated and manual test cases were executed across different system modules. The distribution of test counts, testing frameworks, and execution results are summarized in Table 4.12.

Table 4.12: Summary of test executions and pass rates

Module	Testing Tool / Framework	Total Test Cases	Passed	Pass Rate
Frontend Unit	Flutter Test (Riverpod, Repositories)	35	35	100%
Frontend UI	Widget & Golden Visual Tests	15	15	100%
Backend Unit	Jest (Services, DTO validators)	17	17	100%
Backend E2E	Supertest (HTTP REST endpoints)	10	10	100%
MQTT worker	Pytest (MQTT packet forwarding)	8	8	100%
Total System	-	85	85	100%

4.4.4 Retrospective: failure analysis and resolutions

During development testing cycles, several test runs initially failed. A retrospective analysis details the root causes and the engineering solutions implemented:

- Background location tracking failures: Android OS power-saving routines put the background location collection to sleep after 10 minutes. This was resolved by wrapping the background isolate inside a native foreground service using the `flutter_background_service` plugin and showing a persis-

tent notification.

- State synchronization delay in SOS handling: When a rescuer accepts a victim's request, a handling confirmation message is sent to the MQTT worker. However, due to network transit time, the victim's device may not receive this confirmation immediately and continues to broadcast active SOS signals. As a result, the victim's distress pin fails to disappear from the rescuers' map interfaces. This was resolved by implementing a short-term caching mechanism on the MQTT worker: when a "handled" event for Victim X is processed, their ID is temporarily cached for 10 seconds. If any further SOS packets are received from Victim X during this time window, the worker intercepts the packet and mutates the `sos` status flag to `false` before forwarding it, ensuring consistent map UI updates across all clients.

CHAPTER 5. SOLUTION AND CONTRIBUTION

5.1 Key contributions

In addition to delivering a comprehensive system that supports both emergency rescue operations and charitable campaign management, this project provides two notable technical contributions that directly address critical challenges encountered in real-world deployment scenarios. The first contribution is the design and implementation of an MQTT topic architecture, together with optimized publish/subscribe communication rules specifically tailored for location sharing and rescue coordination in environments with unstable Internet connectivity. The second contribution is the modeling and management of the lifecycle states of charitable campaigns, establishing a digital governance framework that helps prevent fraudulent activities. The following sections provide a detailed analysis of these contributions.

5.1.1 MQTT topic architecture and communication mechanism for emergency rescue operations

The first major technical contribution lies in the messaging architecture, which was designed to overcome the limitations of weak Internet connectivity commonly found in flood-affected areas. The diagram in the figure 5.1 illustrates the topic network developed for location-sharing functionality among connected users (e.g., friends and family members). Rather than establishing direct communication between end-user mobile devices (Client A communicating individually with Clients B, C, and D), which may result in increased packet loss and inefficient bandwidth utilization, the system employs an MQTT Client operating as an MQTT Worker within a strong Internet connection environment to serve as an intermediary coordinator. When Client A publishes its current coordinates (*myCurrentLocation*) to the `current-location` topic, the MQTT Worker receives, processes, and redistributes the location data by publishing it to personalized topics for each recipient, such as `A/to_B/last-Location` and `A/to_C/last-Location`. A key innovation of this design is the utilization of MQTT Retained Messages for destination topics. As a result, even if Clients B, C, and D temporarily lose network connectivity, they immediately receive the latest available location of Client A upon reconnection, without waiting for the next update cycle. This mechanism ensures the continuity and reliability of location information dissemination.

5.1.2 Charity campaign lifecycle state management

While the MQTT-based communication architecture focuses on ensuring reliable rescue coordination under unstable network conditions, the second major technical contribution addresses a different challenge: establishing transparency and accountability in charitable activities. To achieve this objective, the system models each charity campaign as a finite-state machine (FSM), as illustrated in the figure 5.3. By defining explicit states and controlled transitions, the system ensures that every campaign follows a predefined lifecycle, thereby reducing opportunities for fraud and improving operational governance.

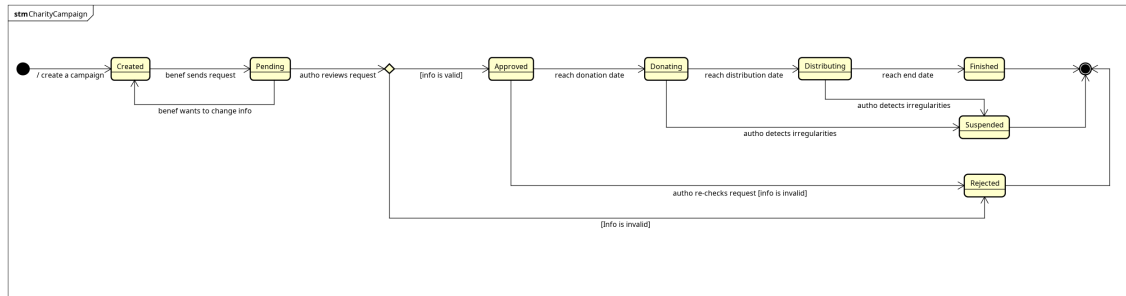


Figure 5.3: Finite state machine diagram for charity campaign

The lifecycle begins with the *Created* state, which is entered immediately after a beneficiary creates a new campaign. At this stage, campaign information is still editable. Once the beneficiary submits the campaign for review, the campaign transitions to the *Pending* state, indicating that it is awaiting verification. If the beneficiary wishes to modify any information during the review process, the campaign can return to the *Created* state for further editing and resubmission.

The transition from *Pending* is governed by an automated verification process. When the system determines that all required information is valid and complete, the campaign is promoted to the *Approved* state. Conversely, if the submitted information is found to be invalid, incomplete, or inconsistent, the campaign transitions to the terminal *Rejected* state.

Once approved, the campaign progresses automatically according to predefined scheduling milestones. Upon reaching the donation start date, the campaign enters the *Donating* state, during which donors are allowed to contribute funds. When the distribution date is reached, the campaign transitions to the *Distributing* state, indicating that collected resources are being allocated to beneficiaries. Finally, when the campaign reaches its designated end date, it moves to the *Finished* state, marking the successful completion of all fundraising and distribution activities.

In addition to the normal workflow, the state model incorporates governance mechanisms to ensure campaign transparency. During the *Donating* and *Distributing* phases, the responsible authority is granted the ability to review campaign activities and supporting information. If irregularities, inconsistencies, or suspicious behaviors are identified during the review process, the authority may transition the campaign to the *Suspended* state. This action temporarily halts all fundraising and distribution activities until the identified issues have been investigated and resolved.

A notable advantage of this state-machine-based approach is that it transforms campaign management from a collection of loosely controlled operations into a structured and auditable process. Every state transition is triggered by a clearly defined business event, such as beneficiary submission, automated validation, scheduled milestones, or fraud detection. Consequently, the system establishes a digital governance framework that enhances transparency, minimizes human error, and significantly reduces the risk of fraudulent charitable campaigns.

5.2 Limitations of the work

Although the FloodHelper system successfully implements the major functionalities proposed in this thesis, several limitations remain.

First, the current implementation supports only Android devices. Users of other mobile platforms, such as iOS, are not supported in the current version.

Second, the real-time location-sharing mechanism relies on Internet connectivity and MQTT communication. When Internet access is unavailable, users cannot continuously synchronize their locations or receive real-time emergency updates.

Third, the charity campaign module currently provides basic functionalities for campaign creation, donation management, and financial transparency. However, it does not support integration with online payment gateways, requiring users to perform bank transfers manually.

Finally, the system has been validated through functional testing in a development environment. Comprehensive evaluations involving large-scale concurrent users, long-term system stability, and performance under disaster-scale workloads have not yet been conducted.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design and implementation of FloodHelper, a disaster assistance platform that integrates real-time location sharing, emergency rescue coordination, and charity campaign management. The system was developed to improve communication and coordination among citizens, rescuers, benefactors, authorities, and administrators during disaster situations through a unified mobile and web-based platform.

Based on the requirements analysis, the system was designed using a hybrid software architecture that combines the Client–Server model with the Publish–Subscribe communication pattern. The mobile application provides functionalities for user management, friend management, location sharing, emergency distress broadcasting, rescue coordination, and charity campaign participation, while the web application supports administrative and authority-related operations. The backend system, together with the MQTT communication service, enables both conventional request-response processing and real-time data synchronization.

The implementation successfully realizes the major functional requirements identified in the analysis phase. Users can register and manage their accounts, establish friendships, share their real-time locations with authorized friends, broadcast distress signals, participate in rescue activities, and contribute to charity campaigns. Authorities are able to review role applications and campaign requests, while administrators manage user accounts and maintain overall system operation. The integration of these modules provides a unified platform that supports multiple disaster-related activities within a single application.

Overall, the implemented system demonstrates the feasibility of integrating social networking, emergency response, and charity campaign management into a unified disaster assistance platform. The adopted system architecture provides a modular and maintainable foundation that can be further extended to support additional functionalities and larger deployment environments in future development.

6.2 Future work

Several directions can be considered to further improve the FloodHelper system.

First, future development may extend the mobile application to additional platforms, such as iOS, allowing a wider range of users to access the system.

Second, the real-time communication mechanism can be enhanced to better

support unstable network conditions by incorporating offline synchronization or delay-tolerant communication techniques for disaster scenarios.

Third, the charity campaign module can be integrated with online payment gateways to simplify the donation process and provide a more convenient user experience.

Finally, future work may include comprehensive performance, scalability, and usability evaluations involving larger numbers of users and more realistic disaster simulation scenarios. Such evaluations would provide additional evidence of the system's effectiveness and support its deployment in practical environments.

REFERENCE

- [1] B. VietnamNet, *Thiên tai 2025 phá vỡ hàng loạt kỷ lục, gây thiệt hại 97.000 tỷ đồng*, 2025. [Online]. Available: <https://vietnamnet.vn/thien-tai-2025-pha-vo-hang-loat-ky-luc-gay-thiet-hai-97-000-ty-dong-2469263.html>.
- [2] VNG Corporation, *Zalo - messaging and calling application*, 2026. [Online]. Available: <https://play.google.com/store/apps/details?id=com.zing.zalo&hl=vi>.
- [3] Engineers in FPT Quy Nhơn AI, *An Toàn Mùa Lũ - emergency information and flood support web portal*, 2026. [Online]. Available: <https://antoanmualu.org/>.
- [4] LINQ INC., *Whoo - real-time location sharing social application*, 2026. [Online]. Available: <https://play.google.com/store/apps/details?id=app.whoo&hl=en>.
- [5] Google LLC, *Flutter - build apps for any screen*, 2026. [Online]. Available: <https://flutter.dev/>.
- [6] OpenStreetMap contributors, *OpenStreetMap geographic database*, 2026. [Online]. Available: <https://www.openstreetmap.org/>.
- [7] NestJS, *NestJS - a progressive node.js framework*, 2026. [Online]. Available: <https://nestjs.com/>.
- [8] Node.js Association, *Node.js javascript runtime*, 2026. [Online]. Available: <https://nodejs.org/en>.
- [9] Express.js, *Express - fast, unopinionated, minimalist web framework for node.js*, 2026. [Online]. Available: <https://expressjs.com/en/>.
- [10] The PostgreSQL Global Development Group, *PostgreSQL: The world's most advanced open source relational database*, 2026. [Online]. Available: <https://www.postgresql.org/>.
- [11] Oracle Corporation, *MySQL - the world's most popular open source database*, 2026. [Online]. Available: <https://www.mysql.com/>.
- [12] Prisma, *Prisma - next-generation node.js and typescript orm*, 2026. [Online]. Available: <https://www.prisma.io/>.
- [13] TypeORM, *TypeORM - amazing orm for typescript and javascript*, 2026. [Online]. Available: <https://typeorm.io/>.
- [14] OASIS, *MQTT - the standard for iot messaging*, 2026. [Online]. Available: <https://mqtt.org/>.

- [15] EMQ Technologies Co., Ltd., *EMQX Cloud - fully managed cloud mqtt service*, 2026. [Online]. Available: <https://www.emqx.com/en>.
- [16] Eclipse Foundation, *paho-mqtt - eclipse paho mqtt python client library*, 2026. [Online]. Available: <https://pypi.org/project/paho-mqtt/>.
- [17] Cloudinary, *Cloudinary - image and video management service*, 2026. [Online]. Available: <https://cloudinary.com/>.
- [18] Google LLC, *Firebase Cloud Messaging (FCM) documentation*, 2026. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging?hl=vi>.
- [19] Redis Ltd., *Redis - the real-time data platform*, 2026. [Online]. Available: <https://redis.io/>.
- [20] Docker Inc., *Docker - container application platform*, 2026. [Online]. Available: <https://www.docker.com/>.
- [21] Render Services Inc., *Render - cloud application hosting platform*, 2026. [Online]. Available: <https://render.com/>.
- [22] Microsoft Corporation, *Visual Studio Code - code editing. redefined*, 2026. [Online]. Available: <https://code.visualstudio.com/>.
- [23] Git Project, *Git - distributed version control system*, 2026. [Online]. Available: <https://git-scm.com/>.
- [24] GitHub Inc., *GitHub - where the world builds software*, 2026. [Online]. Available: <https://github.com/>.
- [25] Postman Inc., *Postman api platform*, 2026. [Online]. Available: <https://www.postman.com/>.
- [26] Y. Moon, D. Kim, Y. Go, Y. Kim, Y. Yi, and S. Chong, “Practicalizing delay-tolerant mobile apps with cedos,” in *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '15), Florence, Italy*, 2015, pp. 283–296.
- [27] B. T. Phong, *Thiếu công khai, minh bạch trong hoạt động từ thiện: Lòng tin của công chúng bị xói mòn*, 2025. [Online]. Available: <https://tienphong.vn/thieu-cong-khai-minh-bach-trong-hoat-dong-tu-thien-long-tin-cua-cong-chung-bi-xoi-mon-post1722868.tpo>.
- [28] N. Duy, *Free-gis-data: A repository for free GIS data sources*, 2026. [Online]. Available: <https://github.com/nguyenduy1133/Free-GIS-Data/tree/main>.

- [29] EMQ Technologies Co., Ltd., *EMQX cloud documentation*, 2026. [Online]. Available: <https://docs.emqx.com/en/cloud/latest/>.
- [30] OpenWeather Ltd., *OpenWeatherMap api documentation and services*, 2026. [Online]. Available: <https://openweathermap.org/>.
- [31] VietQR, *VietQR api services and documentation*, 2026. [Online]. Available: <https://api.vietqr.vn/>.